Supporting Information for "Reliability screening for vehicle routing with time windows: a solver-independent alternative to travel-time padding"

Suhan Ali

This document gives the full model, the design of every study, every result table and the further exploratory checks. All numbers are generated from the records in the archive (Ali, 2026d).

Contents

S1 Problem definition **3**

S2 Candidate objectives and scaling **4**
S2.1 Clock-aware approximation and deterministic guarantee 4

S3 Time-dependent stochastic evaluation **5**
S3.1 FIFO travel times 5
S3.2 Correlated disturbances 5

S4 Screening and independent validation **6**
S4.1 Screened conservative speed and reachability-capped contraction 7

S5 Experimental design **7**

S6 First confirmatory study and weight-sensitivity design **9**

S7 Second confirmatory study design **10**

S8 Robust optimization comparator design **11**

S9 Third protocol: robust comparison and fallback rule **12**

S10 Results of every study **13**
S10.1 Complete procedures on matched instances 13
S10.2 Resources on the common subset 14
S10.3 Screening a replaceable generator and capping contractions 14
S10.4 Clock-aware approximation 15
S10.5 Matched disturbance sensitivity 15
S10.6 Diagnosis of screening failures 16
S10.7 First confirmatory study 17

S10.8 Sensitivity to the objective weights . . . 18
S10.9 Solomon instances . . . 19
S10.10 Second confirmatory study . . . 20
S10.11 Gehring–Homberger instances . . . 21
S10.12 Robust optimization comparator . . . 21
S10.13 Robust comparison on the confirmatory instances . . . 23
S10.14 Fallback rule . . . 24
S10.15 Further checks . . . 25

S11 Reproducibility and implementation checks **25**

S12 Origin of the weight vector **27**

S13 Additional outcomes and failure records **27**
S13.1 Instance-level failure audit . . . 30

S14 Further exploratory checks **31**
S14.1 Family-wise screening . . . 31
S14.2 Computation time and buffer choice . . . 31
S14.3 Misspecified delay models . . . 32

S1 Problem definition

Let $N = \{0, 1, \ldots, n\}$ be the nodes, with depot 0 and customers $C = N \setminus \{0\}$. Let V be a homogeneous fleet. Customer i has demand δ_i, deterministic service time s_i, and offered service-start window $[l_i, u_i]$. Vehicle capacity is Q, and the maximum shift duration is $H^{\max}$. The nominal distance and planning time on arc (i, j) are d_{ij} and $\bar{t}_{ij}$.

The planning variables are $x_{ijv} \in \{0,1\}$ for arc use, $z_{iv} \in \{0,1\}$ for customer assignment, $y_v \in \{0,1\}$ for vehicle use, service-start time a_{iv}, depot departure d_v, depot return r_v, and slack deficit $G_{iv} \geq 0$. The assignment variables give each customer one vehicle and activate the corresponding customer–vehicle copy:

$$\sum_{v \in V} z_{iv} = 1 \qquad i \in C, \tag{S1}$$

$$\sum_{j \in N \setminus \{i\}} x_{ijv} = \sum_{j \in N \setminus \{i\}} x_{jiv} = z_{iv} \qquad i \in C,\ v \in V, \tag{S2}$$

$$\sum_{j \in C} x_{0jv} = \sum_{i \in C} x_{i0v} = y_v \qquad v \in V. \tag{S3}$$

Because demand is delivery-only, route capacity needs no cumulative-load variables:

$$\sum_{i \in C} \delta_i z_{iv} \leq Q y_v \qquad v \in V. \tag{S4}$$

The time-propagation constraints below rule out customer-only subtours: summing them around any directed customer cycle would give $0 \geq \sum_i (s_i + \bar{t}_{ij}) > 0$. The argument does not need positive demands. Time windows apply only to assigned copies:

$$l_i z_{iv} \leq a_{iv} \leq u_i z_{iv} \qquad i \in C,\ v \in V. \tag{S5}$$

Separate departure and return variables avoid using one time variable for two depot visits. With a valid global deactivation constant $M \geq H^{\max} + \max_i s_i + \max_{ij} \bar{t}_{ij}$,

$$a_{jv} \geq d_v + \bar{t}_{0j} - M(1 - x_{0jv}), \tag{S6}$$

$$a_{jv} \geq a_{iv} + s_i + \bar{t}_{ij} - M(1 - x_{ijv}), \tag{S7}$$

$$r_v \geq a_{iv} + s_i + \bar{t}_{i0} - M(1 - x_{i0v}). \tag{S8}$$

The bounds $0 \leq d_v, r_v \leq H^{\max} y_v$ deactivate unused vehicles, and $r_v - d_v \leq H^{\max} y_v$. The code enforces these timing relations through OR-Tools routing dimensions and never builds these time constraints explicitly, so no big-M relaxation or big-M diagnostic is claimed for the code.

For target slack s^*,

$$G_{iv} \geq a_{iv} - (u_i - s^*) z_{iv}, \tag{S9}$$

$$G_{iv} \geq 0. \tag{S10}$$

At the optimum, $G_{iv} = \max\{0, s^* - (u_i - a_{iv})\}$ when $z_{iv} = 1$, and $G_{iv} = 0$ when the customer is not assigned to vehicle v.

S2 Candidate objectives and scaling

The four raw criteria are

$$F_1 = c_f \sum_{v \in V} \sum_{i,j \in N} d_{ij} x_{ijv}, \qquad F_2 = \sum_{v \in V} \sum_{i \in C} G_{iv}, \tag{S11}$$

$$F_3 = \sum_{v \in V} (r_v - d_v), \qquad F_4 = \sum_{v \in V} y_v. \tag{S12}$$

F_3 counts driving, service and waiting time, and each vehicle may leave the depot when it suits the schedule, so no vehicle is charged for waiting at the depot before its first service can start.

Reference values come from four heuristic solves, each minimizing one criterion. For each criterion k, L_k and U_k are the smallest and largest values among these anchor solutions. They are *scaling ranges*, not utopia and nadir points, and need not bound the Pareto frontier. No value is clipped. The normalized score is

$$Z(\mathbf{x}) = \sum_{k=1}^{4} w_k \frac{F_k(\mathbf{x}) - L_k}{\max\{U_k - L_k, 1\}}. \tag{S13}$$

The floor of 1 in the denominator avoids division by zero when a discrete criterion takes the same value in every anchor solution. The baseline weights $\mathbf{w} = (0.5693, 0.2643, 0.1055, 0.0609)$ are a fixed experimental preference vector that defines the candidate-selection objective. They were fixed before any experiment reported here, were not elicited from a fleet operator, and make no claim about the preferences of any industry. No result relies on how they were first obtained. For completeness, Section S12 and `code/ahp_weights.py` record their origin: the principal eigenvector of an Analytic Hierarchy Process matrix (Saaty, 1980) set for an earlier list of criteria, whose third criterion, total distance, later became driver duration without the matrix being revised. Section S10.8 varies the weights directly. The weights shape the candidates and rank the admitted ones, but the admission test and the validation do not use them. Any scalar objective could take the place of Z without changing how reliability is judged; the robust comparison (Section S8) uses distance alone. The results do depend on the objective: with equal weights the advantage over conservative speed fell by more than half (Section S10.8).

In OR-Tools, distance is an arc cost, driver duration is a span cost on each vehicle's time dimension, slack deficit is a soft upper bound at each customer, and fleet use is a fixed cost per vehicle. Integer coefficients are scaled once per criterion. The main planning matrix is nominal and fixed, so the original candidate generator does not use clock-dependent travel times. The *conservative-speed* baseline replaces 40 km/h with 33.3 km/h throughout the planning matrix. This pads every trip by 20%, including off-peak trips, so the comparison does not isolate the value of knowing when the peak occurs.

S2.1 Clock-aware approximation and deterministic guarantee

A separate distance heuristic uses the time of day. For each origin, it freezes the departure time estimated from the previous schedule, integrates (S14) at that time to build the origin's outgoing travel-time row, and solves again. Two passes receive 2.5 seconds each. The first reference schedule is the retained nominal plan, whose solve cost has already been incurred separately. For a customer, the departure estimate is the service start plus the service time. For a depot arc, it is the previous departure for the first customer of each route and the mean of the used depot departures for other customers. Each resulting route sequence is rescheduled with the exact deterministic FIFO profile, and binary search finds its latest feasible whole-minute depot departure.

Routes that fail even when leaving at time zero are rejected. The shortest deterministically feasible output is kept, with duration breaking ties. The method is an approximation that reports its failures; it is not a departure-dependent callback or an exact optimizer for the time-dependent VRPTW.

For a feasible conservative-speed plan, the evaluation speed at $\sigma = 0$ never drops below the planning speed during the planned shift. Starting from the same depot departure, FIFO and induction along each route show that the evaluated service starts and the return happen no later than planned. At $\sigma = 0$, such a plan serves every customer on time by construction. The deterministic feasibility of the clock-aware approximation is also checked before a plan is accepted. Neither property is an experimental reliability gain.

S3 Time-dependent stochastic evaluation

S3.1 FIFO travel times

Partition the day into periods $[T_k, T_{k+1})$ with strictly positive speeds v_k. The deterministic time-dependent travel time $t_{ij}(\tau)$ is the smallest $\Delta \geq 0$ satisfying

$$\int_{\tau}^{\tau+\Delta} v(u)\,du = d_{ij}, \qquad v(u) = v_k \text{ for } u \in [T_k, T_{k+1}). \tag{S14}$$

Because the distance covered grows strictly with time, the arrival map $\tau \mapsto \tau + t_{ij}(\tau)$ is nondecreasing, so FIFO holds.

S3.2 Correlated disturbances

For scenario ω, arc travel is

$$T_{ij}(\tau, \omega) = t_{ij}(\tau)\Omega_{ij}(\omega), \tag{S15}$$

where

$$\Omega_{ij}(\omega) = \exp\{\varepsilon_{ij}(\omega)\}, \tag{S16}$$

$$\varepsilon_{ij}(\omega) = \sigma\left(\sqrt{\rho}\,\xi_0(\omega) + \sqrt{1-\rho}\,\xi_{ij}(\omega)\right). \tag{S17}$$

Here ρ is the correlation between the latent Gaussian log-errors. It is not the Pearson correlation between the multipliers, which is

$$\mathrm{Corr}(\Omega_i, \Omega_j) = \frac{\exp(\sigma^2\rho) - 1}{\exp(\sigma^2) - 1}. \tag{S18}$$

The multiplier has median 1 and mean $\exp(\sigma^2/2)$, so the deterministic profile gives the median travel time, which is slightly below the mean.

Scenario banks do not depend on the order in which routes are evaluated. A common factor is drawn once per scenario, and each directed arc gets its own pseudorandom stream keyed by the bank seed and the arc's endpoints. Plans evaluated on the same bank see identical disturbances on the arcs they share.

S4 Screening and independent validation

For a fixed plan R and scenario ω, let

$$q(R,\omega) = \frac{1}{|C|}\sum_{i\in C}\mathbf{1}\{a_i(R,\omega) \le u_i\} \tag{S19}$$

be the on-time customer proportion. The primary service event is

$$J_\kappa(R,\omega) = \mathbf{1}\{q(R,\omega) \ge \kappa\}, \qquad \kappa = 0.95, \tag{S20}$$

and the target is

$$\Pr[J_\kappa(R,\omega) = 1] \ge \alpha, \qquad \alpha = 0.95. \tag{S21}$$

The stricter probability that no customer in the fleet is late on a given day is also reported. It is not used for admission, because it falls mechanically as the number of customers grows.

For N scenarios and empirical success $\hat{p}$, the one-sided Wilson lower bound at confidence $1-\eta$ is

$$L(\hat{p}) = \frac{\hat{p} + z^2/(2N) - z\sqrt{\hat{p}(1-\hat{p})/N + z^2/(4N^2)}}{1 + z^2/N}, \qquad z = z_{1-\eta}. \tag{S22}$$

Candidate generation contracts only the internal planning deadline, to $[l_i, \max\{l_i, u_i - \beta\}]$. The offered window $[l_i, u_i]$ does not change, and every scenario is scored against the original u_i. For each buffer β, a candidate is generated with the slack-aware objective and assessed on the screening bank. The anchor ranges come from the uncontracted ($\beta = 0$) instance and stay fixed across the buffer search. Among the candidates whose screening lower bound reaches α, the one with the smallest normalized planned score (S13) is selected. In simulation every vehicle leaves at its planned d_v; departure times do not change with the scenario. The selected plan is then evaluated once on a larger, untouched validation bank, so the search never selects on the evidence used for the final report. The lower bound holds for each candidate separately, while the search covers 13 buffer levels; Section S14.1 repeats the selection with a Bonferroni bound over the grid.

A prespecified fallback makes the procedure's output defined on every instance. If no candidate passes screening, the procedure returns the uncontracted slack-aware plan ($\beta = 0$). Validation never changes this choice; it only estimates the performance of the plan already chosen. Results for selected plans alone are always labelled as such, and the main procedure comparison includes all instances.

Three reliability quantities appear in the results, and each answers a different question:

- **Admission criterion.** The Wilson lower bound (S22) on the screening bank, computed for each candidate. A candidate is admitted when it reaches $\alpha = 0.95$.
- **Selected-plan probability.** The validated estimate of $\Pr[J_\kappa = 1]$ for a plan that passed screening, from the untouched bank.
- **Procedure-level probability.** The validated probability averaged over all instances, including those on which screening admitted nothing and the fallback was returned.

The chance constraint (S21) is what screening tests for each admitted plan. The procedure-level average falls below α whenever fallback plans are weaker, so the results report admitted plans and the whole procedure separately. The 95% target is the admission standard for each plan, tested with a per-candidate 95% lower

bound. It applies to admitted plans only; where nothing is admitted, the procedure returns the fallback, which has not passed this test.

S4.1 Screened conservative speed and reachability-capped contraction

Screened conservative speed uses the same buffer grid, admission bank and Wilson threshold, but its candidates minimize distance on the padded matrix. Among the passing plans it chooses the shortest, breaking ties by planned duration and then by buffer. Its fallback is the uncontracted conservative plan, or the retained uncontracted slack-aware plan if no conservative plan was found. The procedure's output is therefore defined on all 53 instances, while the conservative generator itself still finds a plan on 46 of them.

Let $e_i = \max\{l_i, \bar{t}_{0i}\}$ be the earliest direct service time under the nominal matrix. The capped weighted variant uses the internal upper bound

$$u_i^{\mathrm{cap}}(\beta) = \min\{u_i, \max\{l_i, u_i - \beta, e_i\}\}. \tag{S23}$$

This stops contraction alone from making a customer unreachable by a direct trip when it was reachable before. It does not guarantee that a joint routing is feasible or that the admission target is met. When a contracted model is identical to one already solved, its archived candidate is reused; only changed models are solved again. Selection still uses the fixed original scaling ranges, the weighted score and the same fallback.

To diagnose failures, a shortest-path relaxation uses arc cost $\bar{t}_{ij} + s_i$ (with $s_0 = 0$) and allows intermediate customers without capacity or window restrictions. Its depot-to-i distance is a valid lower bound even if rounded distances violate the triangle inequality. A contracted deadline below this bound certifies infeasibility. When there is no certificate, a missing solver output is labelled *no feasible plan found*: reaching the time limit without a plan does not prove infeasibility, and passing the reachability test does not prove feasibility. Solver status information was not kept in the archived runs and is not inferred afterwards.

S5 Experimental design

Synthetic instances place a central depot and uniformly distributed customers on a 100×100 grid. Demand is uniform on 10–49 units, service time is 10 minutes, window width is uniform on 90–149 minutes, capacity is 1000, and the shift limit is 600 minutes. Time zero is 06:00 local time, so clock time is the relative schedule time plus 360 minutes. Table S1 gives the seeded instance sizes, and Table S2 lists every model and simulation constant.

Table S1. Seeded synthetic experimental design.

Customers	Vehicles available	Instances	Seeds
20	6	20	7200–7219
50	14	15	7500–7514
100	26	10	8000–8009
200	40	8	9000–9007

Eight configurations are compared in the development study:

1. **Nominal distance**: fixed 40 km/h planning matrix and a distance-minimizing objective.

Table S2. Planning, disturbance, and statistical parameters.

Parameter	Value
Nominal/free-flow speed $\bar{v}$	40 km/h
Shift and clock origin	600 min, beginning 06:00 local time
Capacity; service time; target slack s^*	1000; 10 min; 15 min
Distance cost c_f	15 cost units/km
Objective weights (w_1, w_2, w_3, w_4)	$(0.5693, 0.2643, 0.1055, 0.0609)$
Integer objective scale	100,000
Planned-shift speed divisors	0.9 (06:00–07:00), 1.2 (07:00–09:00), and 1.0 (09:00–16:00); speed is $\bar{v}$ divided by the divisor
Outside-shift continuation	1.0 to 17:00, 1.3 (17:00–19:00), 0.9 thereafter; used if a stochastic route overruns the shift
Log-error scale σ; latent correlation ρ	0.20; 0.50
Exploratory disturbance grid	$\sigma \in \{0.10, 0.20, 0.30\}$; $\rho \in \{0, 0.5, 0.9\}$
Service fraction κ; target α	0.95; 0.95
Wilson tail probability η	0.05 (one-sided 95% lower bound)
Screening; validation; sensitivity scenarios	1000; 5000; 1000 per cell
Buffer grid	$\beta \in \{0, 5, \ldots, 60\}$ min

2. **Conservative-speed distance**: distance minimization with a uniform 33.3 km/h planning matrix.

3. **Slack-aware**: objective (S13) with $\beta = 0$, which shows the effect of the objective without screening.

4. **Fleet-matched distance**: distance minimization forced to use exactly as many vehicles as the slack-aware plan, starting from its feasible routes.

5. **Original procedure**: the complete weighted internal-buffer search, with selection on 1000 screening scenarios, the prespecified $\beta = 0$ fallback, and assessment on 5000 fresh scenarios.

6. **Screened conservative**: conservative-speed distance candidates with the same admission test and the fallback defined above.

7. **Capped procedure**: the original weighted procedure with customer-specific reachability caps.

8. **Clock approximation**: two matrix updates from estimated departure times, followed by exact deterministic rescheduling.

The nominal shift ends at 16:00. The full-day speed profile continues past that time, because lognormal delays are unbounded and a stochastic route can overrun. Evening periods lie outside the planned shift but stay in the evaluator. Fleet-matched distance starts from the slack-aware routes. Because this could carry over their structure, the starting point is treated as part of that comparator's definition.

The screening grid is $\beta \in \{0, 5, \ldots, 60\}$ minutes. The four single-criterion anchor solves get two seconds each to stabilize the scaling ranges, and each candidate solve gets a five-second OR-Tools budget. If a candidate solve returns no feasible plan, the same five-second call is repeated at most twice more. The first feasible result is kept; there is no best-of-several selection. A new model with a reachability certificate is not solved at all. The clock approximation splits one five-second budget between its two solves. Six experiment workers ran in parallel, and elapsed times and software versions are recorded. Within an instance, every configuration uses the same fresh validation bank.

The reliability outcomes are service-event probability, zero-lateness fleet-day probability, mean on-time fraction, expected number of late customers, mean and 95th-percentile total lateness, and mean route success. The operational outcomes are distance, driver duration and fleet use. Simulated duration under the common evaluation model is also reported, to separate actual route time from each generator's planned F_3. Differences are paired by instance, and intervals across instances are kept separate from binomial Monte Carlo intervals within an instance. Across-instance intervals use a paired nonparametric bootstrap with 20,000 resamples and a fixed analysis seed. A claim that distances are equivalent would need a prespecified equivalence margin, so failing to reject a difference is not treated as proof of equality.

S6 First confirmatory study and weight-sensitivity design

A confirmatory protocol was fixed before any new instance was generated or solved. It sets out four directional hypotheses, their tests and decision rules, a failure policy, and 53 instance seeds equal to the development seeds shifted by 60000. The protocol, the code that runs and analyses it, and a SHA-256 manifest of those files were committed to a public repository (commit `41fae67d9520bd5dd01fcac87e680a5a69325d10`) and archived on Zenodo (Ali, 2026a) before the study was run. The analysis program will not run unless the complete prescribed study is present and every frozen file matches the manifest.

Each hypothesis concerns validated service-event probability on the fresh bank, with the instance as the unit. H1 (primary): on instances where conservative-speed planning returns a plan, the original procedure exceeds it. H2 (primary): on all instances, the capped procedure exceeds the original procedure. H3: screening admits the capped procedure on more instances than the original procedure, tested with an exact McNemar test at the 0.05 level. H4: on the instances of H1, the advantage of the original procedure over conservative speed is larger at $\sigma = 0.30$ than at $\sigma = 0.10$ with $\rho = 0$. H1 and H2 use 97.5% intervals, a Bonferroni adjustment for two primary hypotheses, and each is confirmed only if its paired bootstrap interval lies above zero. H4 uses a 95% interval. H3 and H4 are secondary: they carry less weight than H1 and H2 and cannot rescue an unconfirmed primary hypothesis. Holm's procedure in place of the Bonferroni adjustment would not change either primary decision. H4 compares fixed plans: each plan is screened under the primary setting ($\sigma = 0.20$, $\rho = 0.5$) and then evaluated across the sweep without re-screening. In this paper, a hypothesis is *confirmed* when it meets the decision criterion fixed in its protocol, and nothing more is meant. All three protocols are confirmatory only within the prespecified synthetic setting: their instances come from the generator and disturbance model of the development study, so they test repeatability there and are not external validation. H2 and H3 both measure the effect of capping, one on service and one on admission, so they are two views of one change.

A configuration without a plan scores zero on that instance, in validation and in every sweep cell, so a failure counts against the configuration that failed. A software error is recorded with its traceback, is never scored as routing performance, and blocks the analysis.

Parameters, budgets and scenario counts are those of the development study (Table S2). The study ran on an Apple M1 Pro laptop with ten cores under macOS 26.3.1, with Python 3.12.15, NumPy 2.5.3 and OR-Tools 9.15.6755; the development runs used Python 3.12.14 on Linux. Stage 1 ran the four instance sizes as four parallel processes, where the written protocol lists a sequential loop, and stage 2 used six workers as specified. These are the deviations from the written procedure. The scheduling change affects only the CPU time available to each time-limited solve.

The weight analysis is exploratory. The original procedure was rerun on the development instances with

four weight vectors: the original vector, the original with its two largest weights swapped, a distance-heavy vector and equal weights (Section S10.8). Each rerun regenerates the candidate family, screens it on the same bank and validates the selected plan or fallback on the same fresh bank, with the same five-second budget. The vectors were chosen before the runs, and each was run once; a later bracketing run, also exploratory, repeated the original and equal vectors and added two more (Section S10.8).

The benchmark check is also exploratory. It uses the eight R1 and RC1 Solomon files available in the ML4VRP 2024 competition repository (ML4VRP, 2024), which hosts Solomon's instances (Solomon, 1987). The C1 instances are left out because their horizon of 1236 time units is longer than one working shift. Coordinates are read as kilometres, with the study's rounded Euclidean distances and a planning speed of 40 km/h, so one Solomon time unit equals 1.5 minutes. Ready and due times are multiplied by 1.5 and rounded down, service time becomes 15 minutes, and the capacity of 200 and the 25 vehicles are kept. The shift length is the scaled depot due time, 345 minutes for R1 and 360 minutes for RC1, and the shift starts at 06:00 so that the morning peak falls inside it. Planning times are rounded up to whole minutes, as in the main study. This mapping keeps Solomon's relation between travel time and window width, because one distance unit takes one Solomon time unit to drive at the planning speed. The C1 instances could be fitted by compressing time, but that would change how tight their windows are relative to travel times, so the eight instances are a convenience sample of the R1 and RC1 types. The pipeline code is unchanged; only the instance loader and the parameter constructor are replaced.

S7 Second confirmatory study design

The first confirmatory study left two questions open. Its candidate generator produces weaker routes than the state of the art, so the benefit of screening might shrink with candidates from a stronger search. Its post hoc analysis also suggested that returning the conservative-speed plan when screening admits nothing would raise pooled service (Section S10.7), but that rule was chosen after the results were seen. A second protocol tested both questions on 53 further instances, with seeds equal to the development seeds shifted by 80000.

The second generator is the hybrid genetic search of PyVRP (Vidal, 2022; Wouda et al., 2024), version 0.14.0. It solves the same planning model as the OR-Tools generator: the static matrix at the planning speed, hard and possibly contracted windows, capacity, shift length and fleet size, with each vehicle leaving at the latest feasible time. Its weighted objective keeps the distance, driver-duration and fleet terms with the same coefficients. PyVRP cannot express the slack-deficit term, so that term is left out of generation and kept in candidate selection. Each solve stops after 10,000 iterations and uses the instance seed as its random seed, so each HGS solve is deterministic for given inputs. Both generators share the scaling ranges from the unchanged OR-Tools anchor solves. Those solves are time-limited, so the HGS results are deterministic only once the ranges are fixed. The contraction grid, capped windows, reachability certificates, scenario banks, admission rule and selection score are the same as in the OR-Tools procedure. HGS conservative-speed planning minimizes distance at 33.3 km/h. The OR-Tools half reruns the frozen pipeline of the first study unchanged.

Two fallback rules are compared. The slack-aware rule is the one used in the first study; for HGS it returns the uncontracted weighted plan. The conservative rule returns the same generator's conservative-speed plan when screening admits nothing, or the slack-aware plan when there is no conservative plan. Neither rule changes a screening decision or uses a validation outcome.

The protocol fixes four hypotheses on validated service-event probability, with the instance as the unit. G1

(primary): on instances where HGS conservative-speed planning returns a plan, the HGS capped procedure with conservative fallback exceeds it. G2 (primary): on all instances, the conservative fallback improves the OR-Tools capped procedure. G3 (secondary): the same holds for the uncapped OR-Tools procedure. G4: the difference between the HGS and OR-Tools capped procedures, both with conservative fallback, is estimated with a 95% interval and no directional decision. G1 and G2 use 97.5% intervals and G3 a 95% interval, with the bootstrap, failure policy and decision rules of the first study. The protocol requires the abstract and conclusion to report any unconfirmed primary hypothesis.

The protocol, the HGS code, the analysis program and a SHA-256 manifest covering them and the frozen files of the first study were committed to the public repository (commit `3ba6b71288a2a51872ab6a877fa86eb28ff7671d`) and archived on Zenodo (Ali, 2026b) before any instance of the study was solved. Before the runs began, a gate script checked the manifest, the pushed commit and tag, and that no results existed yet. An earlier draft of the protocol used seeds shifted by 70000. The OR-Tools half was run on those seeds before that draft was timestamped, and its console output, which lists the instances that screening admitted, was seen. That seed set was abandoned in full. Its records are kept in the archive but enter no analysis, the protocol discloses them, and the study used the seeds shifted by 80000 with no other change. The HGS generator was built and tested on the development instances, so its development results are exploratory. The study ran on the same machine and software as the first confirmatory study, with PyVRP 0.14.0, and followed the written procedure without deviation.

S8 Robust optimization comparator design

The reliability baselines in the main studies are all based on padding. To compare the procedure with a method that plans against travel-time uncertainty inside the optimization, a robust VRPTW with budgeted uncertainty was implemented (Bertsimas and Sim, 2004; Agra et al., 2013). On arc (i, j) the travel time lies between the nominal planning time $\bar{t}_{ij}$ and $\bar{t}_{ij} + \hat{t}_{ij}$, with $\hat{t}_{ij} = \lceil \theta \bar{t}_{ij} \rceil$ minutes for a protection level θ. A route must stay feasible, with every service start inside its window and the return inside the shift, whenever at most Γ of its arcs take their upper value. Because vehicles may wait, the worst case is computed exactly by the dynamic programme of Agra et al. (2013). For a route with stops $1, \ldots, m$, let $A_k(g)$ be the latest possible service start at stop k when at most g deviations have occurred, with $A_0(g)$ the depot departure and $A_k(-1) = -\infty$:

$$A_k(g) = \max\left\{ l_k,\ A_{k-1}(g) + s_{k-1} + \bar{t}_{k-1,k},\ A_{k-1}(g-1) + s_{k-1} + \bar{t}_{k-1,k} + \hat{t}_{k-1,k} \right\}, \tag{S24}$$

with $s_0 = 0$. The route is robust feasible if $A_k(\Gamma) \le u_k$ for every stop and the worst-case return, computed in the same way, is at most $H^{\max}$. A backward recursion gives the latest start from which the rest of a route remains feasible, so a prefix and a suffix can be joined, and a customer inserted, with an exact check in $O(\Gamma)$ time. Box uncertainty, in which every arc takes its upper value, is the case where Γ is at least the number of arcs. It amounts to planning on the padded matrix $\bar{t}_{ij} + \hat{t}_{ij}$, so the conservative-speed baseline, which plans at $1/1.2$ of the nominal speed, is close to box uncertainty with $\theta = 0.2$. The two differ slightly because of rounding.

Robust models of this kind meet the same obstruction as internal contraction. Under box uncertainty, a customer that cannot be served on time even by a direct trip at its upper travel time makes every plan infeasible, whatever the fleet. With a smaller budget a detour can be faster in the worst case, so the obstruction

is not certain, but in these runs every smaller budget failed on exactly the instances that box uncertainty failed on (Section S10.12). A second variant therefore applies reachability relief, the robust analogue of capped contraction. The robust deadline of each customer is raised to its worst-case direct start when that start is later than u_i, and every original window must still hold under nominal travel times, so no promise is extended.

Both variants were solved by a large neighbourhood search written for this study. It uses random, worst and related removal, greedy and regret insertion, granular relocate, exchange and 2-opt* moves, and simulated-annealing acceptance, and every move is checked exactly for robust feasibility. The objective is total distance, and the fleet is limited to the available vehicles. Each solve stops after a fixed number of iterations (6000 at 20 and 50 customers, 3000 at 100) with the instance seed, so it is deterministic. With $\Gamma = 0$ the search is a nominal VRPTW heuristic. On the development instances it then matched the PyVRP/HGS distance on 20 of the 20 instances with 20 customers, and was 0.04% and 0.9% longer on average at 50 and 100 customers, with HGS run for 10,000 iterations. The feasibility checks agree with brute-force enumeration of deviation sets in the test suite, and every route of every plan is checked again after the search.

Fifteen robust settings were run in each variant on the development instances with 20, 50 and 100 customers: $\Gamma \in \{1, 2, 3, 5\}$ and box uncertainty, each with $\theta \in \{0.2, 0.4, 0.6\}$. The standard variant was also run with $\Gamma = 0$. The instances with 200 customers were run afterwards with a reduced grid (Section S10.12). Each vehicle departs at the latest time that keeps its route robust feasible. Each plan is validated on the same 5000-scenario bank as every other configuration of its instance, so all comparisons are paired, and a setting without a robust plan scores zero, as under the failure policy. A further configuration, screened robust, passes all plans of one variant for an instance through the screening layer (for the standard variant, including $\Gamma = 0$). It selects the shortest plan whose Wilson lower bound on the screening bank reaches 0.95 and otherwise returns the plan with the highest screening success. The settings were fixed before the runs, but no hypothesis was, so the development comparison is exploratory; part A of the third protocol (Section S9) tests it on other instances.

S9 Third protocol: robust comparison and fallback rule

The third protocol (Ali, 2026c) was publicly timestamped, with a hash manifest of the code that runs and analyses it (commit 21ffaa8eed4e899778e14014b73533fd4d14d865), after the exploratory robust comparison and the post hoc fallback analysis (Section S10.14) and before any of its runs. It has two parts, each with its own hypotheses.

Part A repeats the robust comparison on the confirmatory instances of both earlier studies with 20, 50 and 100 customers, 90 instances in all, using the relieved model, the 15-setting grid and the iteration counts of Section S8. Its comparators are the stored results of those studies, so their validated service on these instances was already published. The hypotheses are the contrasts of the development comparison, and no comparator was chosen on the confirmatory results. R1 (primary): screened robust with relief exceeds the OR-Tools capped procedure in service over all 90 instances. R2 (primary): it exceeds the HGS capped procedure with conservative fallback over the 45 instances of the second study. R3 (secondary): it uses more vehicles than that procedure. R4 (secondary): it is shorter than the relieved setting with box uncertainty and $\theta = 0.6$, the most protective one.

Part B tests the best-screened fallback on 53 new instances (development seeds plus 120000) with the frozen OR-Tools and HGS pipelines of the second study. When screening admits no candidate, this rule returns the candidate of the procedure's own family with the highest screening success, ties broken by

the normalized score and then the smaller buffer; if no candidate has a plan, the stored fallback is kept. F1 (primary): it improves the OR-Tools capped procedure over its slack-aware fallback. F2 (primary): it improves the HGS capped procedure over its conservative fallback. F3 and F4 estimate the pooled service of the two procedures with the new rule. Both parts ran on a two-core Linux machine (Intel Xeon, 2.1 GHz; Python 3.13.16 with the pinned package versions), whereas the earlier studies ran on a macOS machine with Apple silicon and Python 3.12.15. Because OR-Tools solves stop at a wall-clock limit, the hardware can change which routes part B's OR-Tools half finds. Part B ran first: its first stage ran two instance sizes at a time, as the protocol allows, and the later stages ran one at a time. Apart from short table-generation and typesetting jobs at low priority during the first minutes of the first stage, no other computation ran alongside the OR-Tools and HGS runs. Part B took about 2.3 hours of run time and part A, whose robust search is deterministic, about 7.6 hours.

Within each part, the two primary hypotheses use 97.5% paired bootstrap intervals (a Bonferroni adjustment; 20,000 resamples with the seed of the earlier studies), and each is confirmed only if its interval lies above zero. R3 and R4 use 95% intervals; R3 is confirmed if its interval lies above zero and R4 if it lies below zero. F3 and F4 are estimates. Both analysis programs stop unless every record is present and complete and every frozen file matches the manifest, and part B reuses the frozen validators of the second study.

S10 Results of every study

S10.1 Complete procedures on matched instances

Table S3 compares every original configuration on the same subset of instances, those on which conservative-speed planning returned a plan. The paired contrasts are computed from differences within each instance, so their intervals measure the comparison directly; they are not obtained by subtracting the endpoints of separate intervals.

Table S3. Validated service-event probability (%) on the same 46 instances: every entry in a row has the displayed denominator; k is the number of instances per size. Entries are mean [95% instance-bootstrap interval]. N, C, S, F and P denote nominal, conservative-speed, slack-aware, fleet-matched and original procedure, respectively.

n	k	N	C	S	F	P
20	18	21.5 [18.8, 24.4]	64.2 [61.1, 67.4]	53.3 [47.6, 59.4]	21.7 [18.9, 24.8]	85.7 [74.4, 95.0]
50	13	17.7 [16.1, 19.6]	59.0 [54.6, 63.1]	45.8 [43.1, 48.4]	16.9 [14.8, 18.9]	76.8 [62.3, 89.7]
100	8	25.4 [23.5, 27.1]	71.6 [69.1, 74.0]	55.3 [51.5, 59.3]	26.2 [23.6, 29.0]	87.6 [74.2, 97.5]
200	7	26.7 [24.9, 28.8]	74.6 [72.2, 77.0]	61.7 [58.3, 66.8]	27.7 [25.5, 30.6]	82.9 [69.3, 94.3]

Table S4. Paired differences in service probability (percentage points) on the same subsets as Table S3. SC denotes screened conservative speed with its prespecified fallback.

n	S minus C	P minus C	SC minus C
20	−11.0 [−16.1, −6.0]	21.4 [9.6, 31.3]	28.2 [21.4, 34.3]
50	−13.1 [−17.4, −8.9]	17.9 [2.8, 32.0]	11.8 [2.7, 22.2]
100	−16.2 [−20.6, −11.7]	16.0 [4.8, 24.8]	11.2 [2.8, 19.4]
200	−12.9 [−18.5, −5.7]	8.3 [−5.1, 20.0]	5.7 [0.0, 12.9]

In the paired contrasts of Table S4, the complete original procedure exceeds conservative-speed planning

by 8.3–21.4 percentage points, depending on size. The slack-aware objective without screening shows no such advantage. At $n = 200$ the interval includes zero, so the positive mean at that size is not conclusive.

S10.2 Resources on the common subset

Table S5. Paired original-procedure changes relative to conservative speed on the common subset. Distance and duration are mean percentage changes; fleet is an absolute difference. Scheduled duration is each planner's own F_3. Simulated duration uses the same realized traffic model and planned departures for both configurations.

n	Distance	Scheduled time	Simulated time	Fleet
20	4.9 [2.4, 7.3]	0.3 [−3.8, 4.8]	8.5 [4.4, 12.9]	0.22 [0.00, 0.44]
50	2.3 [−0.6, 5.2]	−7.5 [−12.2, −2.7]	−2.0 [−7.0, 2.8]	−0.46 [−1.00, 0.00]
100	2.0 [−1.3, 5.1]	−3.0 [−9.4, 2.5]	2.2 [−4.3, 7.6]	0.12 [−0.38, 0.62]
200	2.4 [−1.4, 5.7]	−7.7 [−15.2, −1.0]	−2.8 [−10.6, 4.1]	−0.71 [−2.57, 1.00]

Scheduled driver duration differs from the realized driving, service and waiting time in simulation: the conservative planner budgets padded arc times, while in simulation those arcs may be driven faster. Table S5 therefore reports both. Absolute distance premiums also depend on solver quality: in the five-second deterministic benchmark, OR-Tools routes are about 11% longer than HGS routes at $n = 200$.

The original procedure does not use fewer resources at every size. At $n = 20$ it has slightly more scheduled time and more vehicles, and at $n = 100$ it also uses slightly more vehicles. The simulated-time interval excludes zero only for the increase at $n = 20$. The service gain is therefore bought with some extra operating resources.

S10.3 Screening a replaceable generator and capping contractions

Table S6. Procedure outcomes on all instances. Selected counts instances where the lower bound of some candidate on the screening bank reached 0.95; retained counts the selections whose lower bound on the fresh validation bank also reached 0.95. Probability includes fallback plans as well as selected plans.

n	Procedure	Selected / instances	Retained / selected	Probability (%)
20	Original procedure	14/20	14/14	81.6 [70.2, 91.9]
20	Screened conservative	15/20	15/15	87.7 [79.0, 94.7]
20	Capped procedure	14/20	14/14	81.6 [70.3, 92.0]
50	Original procedure	8/15	8/8	72.7 [59.2, 86.4]
50	Screened conservative	4/15	4/4	67.5 [58.2, 77.3]
50	Capped procedure	10/15	10/10	80.1 [66.9, 90.8]
100	Original procedure	6/10	5/6	81.3 [68.8, 93.3]
100	Screened conservative	4/10	4/4	77.5 [67.2, 88.2]
100	Capped procedure	10/10	9/10	97.5 [96.7, 98.2]
200	Original procedure	4/8	3/4	80.6 [68.7, 92.3]
200	Screened conservative	2/8	2/2	78.3 [71.0, 86.5]
200	Capped procedure	8/8	7/8	97.6 [96.9, 98.2]

In Table S6, screened conservative speed selects a plan on 25 of 53 instances, and 25 of these selections keep the target lower bound on the fresh bank. Over all instances, its paired difference from the original procedure is −0.3 [−5.5, 5.0] points. The capped variant selects a plan on 42 of 53, with 40 retained, and its

paired difference from the original procedure is 7.7 [3.5, 12.5] points. Capping removes only the reachability obstruction at individual customers; limits from capacity, sequencing and finite search remain.

Table S7. Operating cost of the capped variant relative to the original procedure on all 53 instances. Percentage changes are paired by instance; fleet changes are absolute. Capping can lead to different routes and resources, so its reliability gain is not attributed to schedule structure alone.

n	Distance	Scheduled time	Simulated time	Fleet
20	0.0 [0.0, 0.0]	0.0 [0.0, 0.0]	0.0 [0.0, 0.0]	0.00 [0.00, 0.00]
50	2.2 [0.0, 5.4]	4.6 [0.0, 11.4]	4.5 [0.0, 11.3]	0.33 [0.00, 0.87]
100	4.9 [1.1, 8.9]	6.8 [1.7, 12.1]	6.6 [1.7, 11.8]	0.30 [−0.20, 0.80]
200	6.3 [1.6, 10.9]	10.5 [3.1, 18.0]	10.0 [2.9, 17.1]	1.88 [0.50, 3.38]

Over all instances (Table S7), the capped variant uses 2.5% more distance and 0.43 more vehicles than the original procedure. At $n = 200$ the extra fleet is substantial. The earlier fleet-matched ablation concerns the uncontracted objective and does not cover this cost.

S10.4 Clock-aware approximation

Table S8. Clock-aware approximation. Each row uses the same instances for every column: those where the approximation found an exact deterministic schedule. Probabilities are percentages, and paired changes are percentage points. The two matrix solves share a five-second search budget.

n	Found	Nominal	Clock-aware	Original P	Clock minus N
20	19/20	21.2 [18.6, 24.0]	39.5 [37.9, 41.3]	84.1 [73.2, 93.6]	18.4 [15.1, 21.4]
50	14/15	17.5 [16.0, 19.3]	36.5 [35.4, 37.7]	74.8 [60.8, 87.1]	19.0 [17.2, 20.9]
100	10/10	25.4 [23.9, 26.8]	44.5 [43.0, 46.0]	81.3 [68.8, 93.2]	19.1 [17.6, 20.8]
200	6/8	26.6 [24.5, 28.9]	43.7 [42.3, 45.3]	81.7 [68.7, 93.6]	17.2 [15.7, 18.7]

The clock-aware rows test a practical approximation that uses departure times. Exact deterministic retiming checks every reported route, so its success at $\sigma = 0$ is a condition for acceptance and says nothing about robustness to random delay. Conservative-speed planning also guarantees deterministic on-time service within the planned horizon, because its planned arc times are never shorter than the deterministic ones. Both 100% results hold by construction.

In Table S8, the approximation exceeds nominal distance by 17.2–19.1 points on the instances where it finds a schedule, but stays below the original procedure. The latest-feasible-departure rule and the two-pass search are part of this result, and the comparison does not show what an optimal time-dependent planner could achieve.

S10.5 Matched disturbance sensitivity

The comparison of interest in Table S9 is the paired difference P minus C, which Fig. S1 plots across the disturbance grid. It shows how the same fixed plans compare under each setting. This column alone cannot separate screening from the contracted candidate family, and the nine cells share instances and common random numbers.

For independent disturbances, raising σ from 0.10 to 0.30 moves the paired difference from -3.1 [-10.7, 3.9] to 30.5 [19.8, 40.5] points. This supports a growing advantage of the complete screened procedure in

Table S9. Fixed-plan sensitivity on the 46 common instances. Each cell uses 1,000 common scenarios from a bank separate from the main validation bank; plans are not re-screened or re-optimized for each setting. C, P and SC are conservative speed, the original procedure and screened conservative speed.

σ	ρ	C (%)	P (%)	SC (%)	P minus C (points)
0.10	0.0	91.8 [88.2, 95.0]	88.8 [81.3, 95.3]	97.1 [94.3, 99.2]	−3.1 [−10.7, 3.9]
0.10	0.5	84.6 [81.5, 87.4]	87.8 [81.5, 93.5]	93.2 [90.1, 95.9]	3.2 [−3.3, 9.3]
0.10	0.9	81.2 [78.3, 84.0]	87.6 [81.4, 93.3]	91.5 [88.2, 94.5]	6.4 [0.0, 12.5]
0.20	0.0	69.8 [64.1, 75.5]	81.0 [71.4, 89.7]	86.7 [81.0, 92.0]	11.2 [1.4, 20.6]
0.20	0.5	65.6 [63.2, 67.9]	82.8 [75.9, 89.2]	82.9 [78.0, 87.5]	17.2 [10.3, 23.7]
0.20	0.9	67.2 [65.3, 68.9]	82.5 [76.8, 87.9]	82.1 [77.9, 86.2]	15.4 [9.5, 20.9]
0.30	0.0	41.7 [37.4, 46.0]	72.1 [61.1, 82.4]	69.4 [60.6, 77.8]	30.5 [19.8, 40.5]
0.30	0.5	54.4 [52.5, 56.3]	74.0 [67.8, 79.8]	72.1 [67.1, 77.0]	19.6 [13.6, 25.3]
0.30	0.9	60.6 [59.3, 61.9]	75.6 [70.7, 80.1]	74.4 [70.6, 78.2]	15.0 [10.0, 19.6]

that setting. At $\rho = 0.9$ the increase is not monotonic, so the data do not show that the value of screening always rises with the noise scale.

S10.6 Diagnosis of screening failures

Of the 21 instances on which the original procedure admitted no plan, 17 reach a buffer at which some customer is provably unreachable, and 4 have no such certificate up to 60 minutes. On 8 instances the solver also returned no plan at a buffer not covered by a certificate. Those outcomes remain unresolved: failing a necessary reachability test proves infeasibility, but passing it does not show that a feasible multi-customer routing exists. Appendix S13.1 gives the audit for each instance.

A further exploratory check took the 8 failures whose search ended with no plan and no reachability certificate, and solved exactly that contracted model again with a longer budget: 30 s, then 120 s for any still without a plan. In 4 cases a plan was found at that contraction level, so the original failure was a search limit. The remaining 4 still had no plan after 120 s, which does not prove infeasibility. The script and records are in the reproducibility archive (Ali, 2026d).

S10.7 First confirmatory study

The four hypotheses of the prespecified protocol were tested on 53 new instances (Table S10). Every decision uses the unrounded interval limits, and the protocol's validation checks passed before any quantity was computed.

Table S10. First confirmatory study on 53 new instances. Differences are in percentage points of validated service-event probability; intervals are paired bootstrap intervals over instances. Conservative speed returned a plan on 50 of 53 instances, and H1 and H4 use those instances, as the protocol specifies.

Hypothesis	Quantity	Estimate [interval]	Level	Decision
H1 (primary)	Original procedure minus conservative speed, 50 instances	15.7 [7.5, 23.6]	97.5%	confirmed
H2 (primary)	Capped minus original procedure, 53 instances	11.7 [5.9, 17.9]	97.5%	confirmed
H3	Admitted by capped / original procedure	46 / 32 (capped only 14, original only 0)	exact $p = 0.00012$	confirmed
H4	Advantage at $\sigma = 0.30$ minus at $\sigma = 0.10$ ($\rho = 0$), 50 instances	28.3 [19.6, 36.4]	95%	confirmed

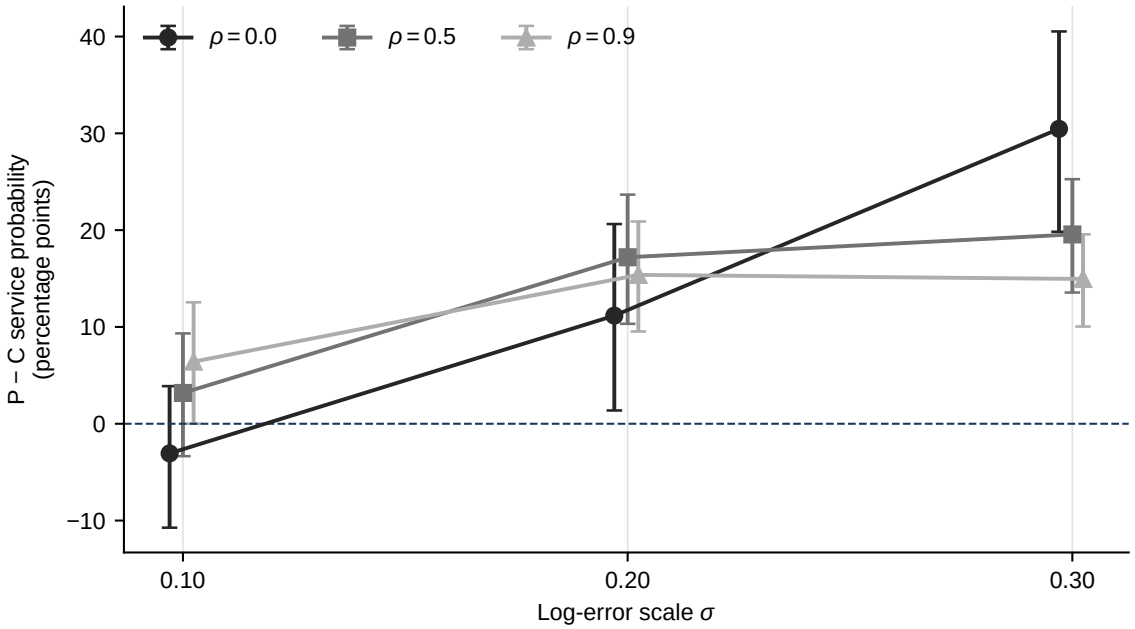


Figure S1. Paired difference in service probability between the original procedure (P) and conservative-speed planning (C) on the 46 common instances. Circles, squares and triangles mark $\rho = 0$, 0.5 and 0.9, and the dashed line marks zero. Faint vertical lines mark $\sigma = 0.10$, 0.20 and 0.30; markers are offset slightly around them only for legibility. Error bars are 95% bootstrap intervals over instances. Plans are fixed across this disturbance sweep, and the intervals are not simultaneous across cells.

All four hypotheses met their protocol-defined decision criteria. The original procedure exceeded conservative speed by 15.7 points, and capping added 11.7 points. Capping admitted 46 instances against 32 and lost none that the original procedure admitted (14 gained, 0 lost). For independent disturbances, the advantage over conservative speed was 28.3 points larger at $\sigma = 0.30$ than at $\sigma = 0.10$. Of the plans admitted on the screening bank, 31 of 32 (original) and 45 of 46 (capped) kept the 0.95 lower bound on the fresh bank.

Averaged over all instances, service probability was 79.4% for the original procedure, 82.4% for screened conservative speed and 91.0% for the capped procedure, so on average the 95% target is still not met. Screened conservative speed was not a prespecified comparison, and its value is descriptive only. As in the development study, the shortfall comes from instances without an admitted plan. Plans admitted by screening averaged 97.4% for the original procedure and 97.5% for the capped procedure, while the fallback plans averaged 51.9% (21 instances) and 48.7% (7 instances).

H2 and H3 are not independent. When neither procedure admits a plan, both return the same fallback, and 11.6 of the 11.7 points of H2 come from the 14 instances that only the capped procedure admits. The two hypotheses measure one finding twice.

Post hoc fallback. The protocol fixes the fallback: when no plan is admitted, both procedures return the uncontracted slack-aware plan. Returning the conservative-speed plan instead, where that plan exists, would have raised pooled service on the same fresh bank from 79.4% to 85.0% for the original procedure (18 instances changed) and from 91.0% to 93.0% for the capped procedure (6 instances changed). This rule uses no validation outcome, but it was chosen after the results were seen, so the comparison is exploratory. The second confirmatory study tests it prospectively (Section S10.10).

These instances come from the same generator and the same assumed disturbance model as the development study, so the result shows repeatability within that setting. It says nothing about real road networks, calibrated parameters or other instance families.

S10.8 Sensitivity to the objective weights

The original procedure was rerun on the 53 development instances with four weight vectors (Table S11); neither confirmatory study varied the weights. Each rerun regenerates the candidate family, screens it on the same bank and validates the selected plan or fallback on the same fresh bank. Conservative-speed planning does not use the weights, so its archived outcome is reused. The rerun with the original weights differs from the archived run by -1.4 points pooled ([-5.1, 2.2]) and by one admitted instance (31 against 32). This is the run-to-run variation of a time-limited search.

Table S11. Weight sensitivity on the 53 development instances. Weights are ordered distance cost, slack deficit, driver duration, fleet use. The advantage over conservative speed uses the 46 instances where conservative speed returned a plan. The last column is the paired difference in service probability from the rerun with the original weights, over all 53 instances (percentage points).

Vector	Weights	Admitted	Pooled (%)	Advantage over conservative [95%]	Paired difference [95%]
Original (rerun)	(0.5693, 0.2643, 0.1055, 0.0609)	31	77.5	16.2 [9.2, 22.8]	–
Slack first	(0.2643, 0.5693, 0.1055, 0.0609)	30	77.4	16.0 [9.6, 22.1]	-0.1 [-3.9, 3.3]
Distance heavy	(0.8, 0.1, 0.05, 0.05)	29	74.3	12.7 [5.3, 19.7]	-3.2 [-7.1, 0.2]
Equal	(0.25, 0.25, 0.25, 0.25)	23	68.7	6.3 [-1.1, 13.7]	-8.8 [-14.3, -3.5]

Swapping the two largest weights changed nothing beyond the run-to-run variation. The distance-heavy vector gave pooled service 3.2 points lower, with a paired interval that includes zero, and its advantage over conservative speed stayed clearly positive (12.7 [5.3, 19.7]). Equal weights changed the most. They admitted 23 instances against 31, lowered pooled service by 8.8 points (paired interval -8.8 [-14.3, -3.5]; -15.6 to -2.4 after adjusting the level for three comparisons), and left an advantage over conservative speed whose interval includes zero (6.3 [-1.1, 13.7]). Of the instances whose admission differed, 9 were admitted only under the original weights and 1 only under equal weights (exact $p = 0.021$, unadjusted; the adjusted threshold for three comparisons is 0.017).

This analysis is exploratory. It uses the development instances and four vectors, chosen without searching for the point at which the advantage disappears, and each vector was run once; the original and equal vectors were rerun later, below. Equal weights give fleet size and driver duration 0.25 each, against 0.06 and 0.11 in the original vector; the run below separates the two. To find what breaks under equal weights, two further vectors were fixed before a new run on the same instances: the original vector with the fleet-use weight raised to 0.25, and the original vector with the driver-duration weight raised to 0.25, the other weights scaled in proportion (`code/weight_bracketing.py`). Equal weights change both at once. The original and equal vectors were rerun on the same machine, so the four rows of Table S12 are comparable with each other; the run also stored every candidate, which the earlier run did not.

Rerun on the same machine, the original vector admitted 32 instances and the equal vector 24, and the advantage over conservative speed fell from 17.5 [10.8, 23.9] to 8.1 [1.1, 15.1] points. The earlier run (Table S11) gave 6.3 points with an interval from -1.1 to 13.7, so in both runs equal weights cut the advantage

Table S12. Weight bracketing on the 53 development instances (exploratory). Advantage over conservative speed on the instances where it has a plan, paired difference in service from the original vector rerun on the same machine (percentage points), and, averaged over instances, the share of candidates whose screening lower bound reached 0.95 and the mean vehicles per candidate.

Vector	Weights	Adm.	Pooled	Advantage [95%]	Diff. from original	Pass (%)	Veh.
Original (rerun here)	(0.5693, 0.2643, 0.1055, 0.0609)	32	78.7	17.5 [10.8, 23.9]	–	17.3	7.88
Equal (rerun here)	(0.25, 0.25, 0.25, 0.25)	24	70.2	8.1 [1.1, 15.1]	−8.5 [−13.0, −4.5]	9.0	7.24
Fleet weight raised	(0.4546, 0.2111, 0.0843, 0.25)	27	73.2	11.7 [4.7, 18.6]	−5.6 [−10.1, −1.4]	13.5	7.58
Duration weight raised	(0.4773, 0.2216, 0.25, 0.0511)	25	69.9	7.4 [−0.6, 15.4]	−8.9 [−14.1, −4.2]	11.8	7.71

by more than half, but whether its interval reaches zero differs between runs. Raising the driver-duration weight to 0.25, with the other weights scaled down, lowered service about as much as equal weights did (−8.9 [−14.1, −4.2] against −8.5 [−13.0, −4.5] points from the original vector), and raising the fleet weight cost −5.6 [−10.1, −1.4] points. The run cannot separate the two effects: the duration-raised vector minus the fleet-raised one was −3.3 [−10.2, 3.4] points. The stored candidates suggest why the duration weight matters. Compared at the same instance and buffer, the duration-heavy candidates used only 0.18 fewer vehicles, against 0.29 for the fleet-heavy ones, yet their success on the screening bank fell more (2.5 against 1.1 points): they shortened driver duration by 85 minutes per plan on average and raised the slack deficit, so less waiting time was left to absorb delays. A heavier duration weight thus appears to remove slack that screening relies on. Of the 53 runs with the original vector, 39 ran on the first day while other analyses were also running on the machine, and the other runs of the bracketing ran later with the machine otherwise idle; because OR-Tools stops at a wall-clock limit, this may affect the comparison slightly.

S10.9 Solomon instances

The unchanged pipeline was run on eight 100-customer Solomon instances, with the study's time-dependent profile and disturbance model added (Section S6). Within its five-second budget, OR-Tools returned a nominal plan for five of the eight instances (Table S13). PyVRP/HGS finds a feasible plan for the other three, R101, R102 and RC105, within 40 seconds, so these failures are search limits of the generator, not infeasibility of the mapped instances. The failure policy records them as missing plans, and every configuration that depends on them as not attempted.

Of the five instances with a plan, screening admitted a plan on three (R112, RC104 and RC108). Their validated service probability was 97.7 to 99.4%, against 78.0 to 82.0% for conservative speed and 22.8 to 29.4% for nominal distance. On the other two, the procedure returned the slack-aware fallback: R103 (45.1%; conservative speed returned no plan there, although HGS finds a feasible plan on its matrix) and RC106 (42.1%), where it fell 22.9 points below conservative speed (65.0%). Capping changed no result on any instance.

On RC106, screening admitted no plan, and the slack-aware fallback was weaker than the conservative plan. The conservative-speed fallback examined post hoc in Section S10.7 would have returned 65.0% there.

The eight instances are all the R1 and RC1 files in the source repository. They give no basis for an interval estimate, and no Solomon comparison was prespecified. The admitted plans reached 97.7 to 99.4% and the fallback plans 42.1 to 45.1%, a split similar to that on the synthetic instances. The generator, however, returned no plan on a substantial share of these harder instances.

Table S13. Solomon instances: validated service-event probability (%). A dash marks a configuration with no plan. *Plan* states whether OR-Tools returned a nominal plan; *Admitted* states whether screening admitted a plan for the original procedure, which otherwise returns the slack-aware fallback.

Instance	Plan	Nominal	Conservative	Slack-aware	Original	Capped	Admitted
R101	no	–	–	–	–	–	no
R102	no	–	–	–	–	–	no
R103	yes	14.5	–	45.1	45.1	45.1	no
R112	yes	29.4	78.0	53.7	99.4	99.4	yes
RC104	yes	22.8	82.0	68.9	97.7	97.7	yes
RC105	no	–	–	–	–	–	no
RC106	yes	19.9	65.0	42.1	42.1	42.1	no
RC108	yes	28.3	80.7	51.5	98.8	98.8	yes

S10.10 Second confirmatory study

In the exploratory run on the 53 development instances, the HGS capped procedure with conservative fallback exceeded HGS conservative-speed planning by 27.0 points, with pooled service of 94.2%. Table S14 gives the prespecified results on the 53 new instances of the second study.

Table S14. Second confirmatory study on 53 new instances. Differences are in percentage points of validated service-event probability; intervals are paired bootstrap intervals over instances. G1 uses the 49 instances where HGS conservative-speed planning returned a plan.

Hypothesis	Quantity	Estimate [interval]	Level	Decision
G1 (primary)	HGS capped with conservative fallback minus HGS conservative, 49 instances	28.0 [23.0, 32.8]	97.5%	confirmed
G2 (primary)	OR-Tools capped, conservative minus slack-aware fallback, 53 instances	0.4 [−0.7, 2.1]	97.5%	not confirmed
G3	OR-Tools uncapped, conservative minus slack-aware fallback, 53 instances	3.8 [1.6, 6.2]	95%	confirmed
G4	HGS minus OR-Tools, both capped with conservative fallback, 53 instances	−0.2 [−4.0, 3.7]	95%	estimate only

G1 was confirmed. With the HGS generator, screening with capped contraction exceeded conservative speed by 28.0 points, and pooled service probability reached 91.6% against 58.4% for HGS conservative speed. Capping was essential with this generator: at 100 and 200 customers the HGS capped family was admitted on 18 of 18 instances, and the uncapped family on 3.

G2 was not confirmed. The OR-Tools capped procedure admitted a plan on 45 of 53 instances and fell back on 8. On 4 of those no conservative-speed plan existed, so both rules returned the same plan. On the remaining 4 the conservative fallback changed service probability by +5.4, −3.1, −12.0 and +30.0 points. The post hoc gain seen for the capped procedure in the first study did not replicate. For the uncapped procedure, which falls back more often, the conservative fallback helped (G3, 3.8 points).

Under the capped procedure with conservative fallback, the two generators differed by −0.2 points (G4), with a 95% interval from −4.0 to 3.7. No equivalence margin was prespecified, so this interval bounds the difference but does not establish equivalence. The HGS $\beta = 0$ plan, which lacks the slack term, was a

weak fallback: the uncapped HGS procedure pooled 51.0% with it and 74.7% with the conservative fallback (descriptive).

S10.11 Gehring–Homberger instances

As a harder external check, the HGS pipeline of the second study was run on the 20 Gehring–Homberger instances of classes R1 and RC1 with 200 customers (Gehring and Homberger, 1999) (`code/gh_benchmark.py`; exploratory). The mapping follows the Solomon check, except that the depot horizon of 634 time units is scaled to the 600-minute shift and the coordinates are scaled so that the benchmark's unit-speed travel corresponds to 40 km/h; capacity and fleet are kept. On R1_2_1 and R1_2_2, where all or most windows are only 10 time units wide, neither the OR-Tools anchor solves nor HGS at conservative speed found a plan, so the procedure could not run there. Screening admitted a plan on 17 of the other 18 instances (Table S15). Where conservative speed had a plan, the HGS capped procedure exceeded it by 15.1 [11.0, 19.1] points (18 instances), and over all 20 instances it pooled 87.2% against 73.6% for conservative speed and 34.9% for the nominal plan.

Table S15. Gehring–Homberger instances with 200 customers (exploratory, HGS pipeline): instances, instances where the anchor solves found a plan, instances admitted by screening, and pooled validated service-event probability (%) of the HGS nominal plan, HGS conservative speed and the HGS capped procedure with conservative fallback. A missing plan scores zero.

Class	Inst.	Anchors	Admitted	Nominal	Conservative	Capped
R1	10	8	7	42.1	68.0	77.0
RC1	10	10	10	27.6	79.3	97.4

S10.12 Robust optimization comparator

Table S16 compares the robust plans with the screened procedures on the 45 development instances with 20, 50 and 100 customers, and Fig. S2 shows the trade-off between service and distance; the appendix lists every setting. Every distance, including those of the OR-Tools and HGS plans, is given relative to the nominal plan found by the robust search with $\Gamma = 0$; the OR-Tools nominal plans are 1.7% longer on average.

The standard robust model fails on the larger instances. Under box uncertainty it has no robust plan on 8 of the 45 instances with $\theta = 0.4$ and on 16 with $\theta = 0.6$. These are exactly the instances on which some customer cannot be served on time even by a direct trip at its upper travel time, so no fleet size helps. Every smaller budget Γ failed on exactly the same instances. This is the reachability obstruction that capped contraction addresses (Section S4.1). The mildest box setting, $\theta = 0.2$, the robust counterpart of conservative speed, still had no plan on 4 instances and pooled 60.4%.

Reachability relief removes the obstruction. Every relieved setting has a plan on all 45 instances, and under box uncertainty with $\theta = 0.4$ relief raised pooled service by 16.4 points (Table S17). On the 37 instances where the standard model had a plan, the two models returned the same routes, so the whole gain comes from the instances without a standard robust plan. The relieved robust plans then compete with the screened procedures. With $\Gamma = 3$ and $\theta = 0.4$ they exceeded the OR-Tools capped procedure by 4.8 points on average, with an interval from -1.4 to 11.6, and used 0.42 more vehicles per instance. Against the HGS capped procedure the protection level decides the comparison. With $\Gamma = 3$, the $\theta = 0.4$ setting was 4.2 points

Table S16. Robust comparator on the 45 development instances (exploratory): number of instances with a plan, mean validated service-event probability (%) by size and pooled, mean distance relative to the nominal plan of the robust search ($\Gamma = 0$) and mean vehicles, both over instances with a plan. A missing plan scores zero. Relief denotes the robust model with reachability relief.

Configuration	Plans	$n = 20$	$n = 50$	$n = 100$	Pooled	Dist. (%)	Veh.
Conservative speed (OR-Tools)	39	57.8	51.1	57.2	55.5	4.8	6.56
Capped procedure (OR-Tools)	45	81.6	80.1	97.5	84.6	9.3	6.76
Capped procedure, conservative fallback (HGS)	45	90.5	94.8	98.0	93.6	12.5	7.11
Robust, box, $\theta = 0.4$	37	90.3	72.5	47.9	75.0	7.0	6.54
Robust, box, $\theta = 0.6$	29	93.0	46.1	29.9	63.3	10.8	6.59
Relief, $\Gamma = 3$, $\theta = 0.4$	45	89.2	87.5	92.5	89.4	7.1	7.18
Relief, box, $\theta = 0.4$	45	90.3	90.0	95.7	91.4	7.6	7.31
Relief, $\Gamma = 3$, $\theta = 0.6$	45	97.6	97.5	98.7	97.8	11.4	7.78
Relief, box, $\theta = 0.6$	45	97.8	98.4	99.4	98.4	12.4	8.04
Screened robust	45	97.0	82.8	75.1	87.4	7.9	7.20
Screened robust with relief	45	97.2	97.4	97.2	97.2	10.4	7.64

less reliable (interval -7.2 to -0.7), with 4.6% less distance and 0.07 more vehicles, while the $\theta = 0.6$ setting was 4.2 points more reliable (interval 1.3 to 7.7) and used 0.67 more vehicles.

Table S17. Paired contrasts on the 45 development instances (exploratory). Service differences are in percentage points over all instances; distance differences are percentage changes and vehicle differences are means, both over instances where both configurations have a plan. Intervals are 95% paired bootstrap intervals over instances.

Contrast	Service [95%]	Distance (%) [95%]	Vehicles
Relief minus standard, box, $\theta = 0.4$	16.4 [6.3, 27.1]	0.0 [0.0, 0.0]	0.00
Relief, $\Gamma = 3$, $\theta = 0.4$, minus OR-Tools capped procedure	4.8 [−1.4, 11.6]	−1.7 [−3.4, 0.1]	0.42
Relief, $\Gamma = 3$, $\theta = 0.4$, minus HGS capped procedure	−4.2 [−7.2, −0.7]	−4.6 [−6.0, −3.1]	0.07
Relief, $\Gamma = 3$, $\theta = 0.6$, minus HGS capped procedure	4.2 [1.3, 7.7]	−0.8 [−2.2, 0.8]	0.67
Screened robust with relief minus HGS capped procedure	3.6 [0.6, 7.1]	−1.6 [−3.0, −0.2]	0.53
Screened robust with relief minus relief, box, $\theta = 0.6$	−1.1 [−1.5, −0.8]	−1.8 [−2.3, −1.2]	−0.40

The screening layer can also wrap the robust generator. Screened robust with relief was admitted by screening on 44 of 45 instances and pooled 97.2%, 3.6 points above the HGS capped procedure (interval 0.6 to 7.1), with 0.53 more vehicles. Against the most protective relieved setting it gave up 1.1 points and saved 1.8% of distance, because it selects the shortest plan that passes the screen. Without relief, screening admitted a robust plan on 29 instances and pooled 87.4%.

These development results are exploratory, and part A of the third protocol tests them below. The settings in Table S16 were picked after the grid was run, to show the trade-off: $\Gamma = 3$, a middle budget, and box uncertainty, the most protective, each at two protection levels; the appendix reports every setting. The robust plans minimize distance alone, without the slack, duration and fleet terms of the screened procedures, so differences in vehicles are part of the comparison.

With 200 customers, the relieved model was run on the 8 development instances with a reduced grid: $\Gamma = 3$ and box uncertainty, each at $\theta = 0.4$ and 0.6, with 2000 iterations (median 114 s per solve). At $\theta = 0.4$

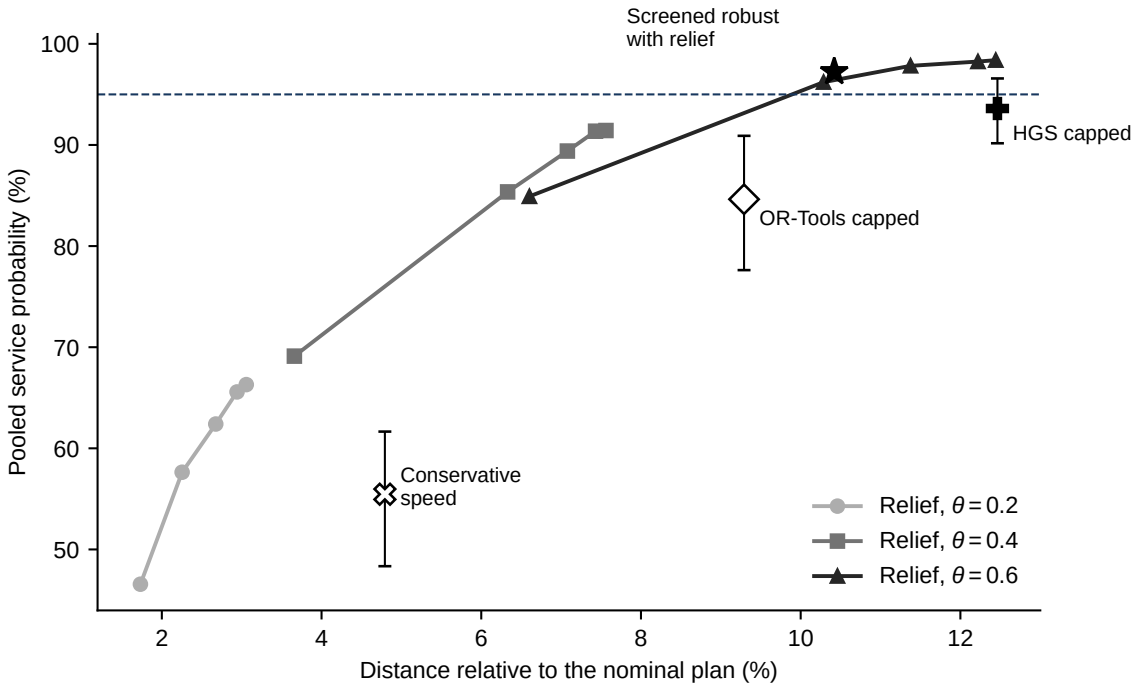


Figure S2. Pooled validated service-event probability against mean distance relative to the nominal plan of the robust search, on the 45 development instances. Lines join the relieved robust settings for each θ, from $\Gamma = 1$ to box uncertainty. The screened procedures (OR-Tools, HGS), screened robust with relief and conservative speed are shown separately, with 95% bootstrap intervals over instances for their service probability. The dashed line marks the 95% target.

the two budgets pooled 93.2% and 96.1%. At $\theta = 0.6$ they pooled 86.8% and 87.2%, because the search found no plan for one instance under either budget; at this size relief does not guarantee a plan, and the search alone does not show whether that model is infeasible. Screened robust over the four settings admitted a plan on 7 instances and pooled 97.8% with 20.6 vehicles on average, against 97.2% with 17.0 vehicles for the HGS capped procedure and 97.6% with 19.1 for the OR-Tools capped procedure on the same instances. With eight instances these figures are descriptive only.

S10.13 Robust comparison on the confirmatory instances

Part A of the third protocol (Ali, 2026c) tested the development findings on the 90 confirmatory instances with up to 100 customers. Table S18 gives the prespecified results.

All four hypotheses met their decision criteria. Screened robust with relief pooled 97.2%, against 90.5% for the OR-Tools capped procedure and 98.4% for the most protective relieved setting; on the second study the HGS capped procedure pooled 90.6%. Screening admitted a robust plan on 86 of 90 instances.

Restricting the robust plans to the comparator's fleet removed much of their advantage. For each instance, the relieved settings were limited to plans that use no more vehicles than the comparator's plan, and the screened-robust rule was applied within that limit (`code/robust_fleet_matched.py`; exploratory, not part of the protocol). The limit also removes the most protective settings on many instances, so the comparison is not a pure fleet effect. On the development instances the restricted plans differed from the HGS capped procedure by -1.9 [-5.1, 1.5] points in service, with 3.8% less distance, and from the OR-Tools capped procedure by 2.5 [-2.7, 8.1] points on the 43 instances with such a plan; screening admitted a restricted plan

Table S18. Third protocol, part A: prespecified results on the confirmatory instances with up to 100 customers. R1 and R4 use all instances of both studies; R2 and R3 use the second study, where HGS plans exist.

Hypothesis	Quantity	Mean [interval]	Level	Decision
R1 (primary)	Screened robust with relief minus OR-Tools capped, service	6.7 [3.0, 11.1]	97.5%	confirmed
R2 (primary)	Screened robust with relief minus HGS capped, service	6.7 [2.2, 12.2]	97.5%	confirmed
R3	Screened robust with relief minus HGS capped, vehicles	0.42 [0.20, 0.67]	95%	confirmed
R4	Screened robust with relief vs. relief box $\theta = 0.6$, distance (%)	-1.3 [-1.7, -0.9]	95%	confirmed

on 20 and 15 instances. On the confirmatory instances of part A the differences were 3.2 [-1.1, 8.1] points against the HGS capped procedure (45 instances, 28 admitted) and -0.7 [-3.9, 2.7] against the OR-Tools capped procedure (87 instances, 41 admitted). Every interval includes zero, so with no larger fleet than the comparator the robust plans were not measurably more reliable.

S10.14 Fallback rule

When screening admits no candidate, each procedure returns a fixed fallback: the uncontracted slack-aware plan for the OR-Tools procedures and the conservative-speed plan for the HGS procedure. These fallbacks are the main reason the procedures pool below the 95% target. After the development study and the first two confirmatory studies, a third fallback rule was examined: return the candidate of the procedure's own family with the highest screening success (ties by the normalized score, then the smaller buffer). The rule uses only the screening evidence already computed, so the validation bank still judges the returned plan independently, and it changes nothing on instances where a plan was admitted. Table S19 compares the rules on the stored records (`code/fallback_policy.py`).

Table S19. Fallback rules on the stored records (exploratory, post hoc): instances where screening admitted nothing, pooled validated service-event probability (%) over all instances under the stored fallback, the conservative-speed plan and the best-screened candidate, and the paired difference best-screened minus stored with a 95% bootstrap interval (percentage points).

Study	Procedure	Fallback inst.	Stored	Cons.	Best	Best minus stored
Development	Original (OR-Tools)	21	78.9	82.7	91.6	12.7 [7.9, 17.9]
Development	Capped (OR-Tools)	11	86.6	88.7	95.1	8.5 [4.0, 13.7]
Development	HGS capped	5	94.2	94.2	97.2	3.0 [0.6, 5.7]
Confirmatory 1	Original (OR-Tools)	21	79.4	85.0	93.2	13.9 [9.0, 19.1]
Confirmatory 1	Capped (OR-Tools)	7	91.0	93.0	96.6	5.6 [1.9, 9.9]
Confirmatory 2	Original (OR-Tools)	25	77.1	80.9	92.7	15.6 [10.7, 20.8]
Confirmatory 2	Capped (OR-Tools)	8	91.4	91.7	96.2	4.9 [1.8, 8.5]
Confirmatory 2	HGS capped	8	91.6	91.6	96.1	4.6 [1.6, 8.1]

The best-screened rule raised pooled service for every study and procedure, by 3.0 to 15.6 points, and every interval lies above zero. With it the capped procedures pooled 95.1 to 97.2%. On the fallback instances themselves its plans averaged 83.0 to 95.0%, against 45.2 to 62.9% for the stored fallbacks. The conservative-speed fallback helped less. Because the rule was chosen after these results were known, part B of the third

protocol tested it on new instances.

Part B ran the frozen OR-Tools and HGS pipelines on 53 new instances (development seeds plus 120000) under the third protocol (Ali, 2026c). Table S20 gives the prespecified results.

Table S20. Third protocol, part B: prespecified results on the new instances. Differences and pooled service are in percentage points and percent; intervals are paired bootstrap intervals over instances.

Hypothesis	Quantity	Mean [interval]	Level	Decision
F1 (primary)	OR-Tools capped, best-screened minus slack-aware fallback	7.3 [2.3, 13.2]	97.5%	confirmed
F2 (primary)	HGS capped, best-screened minus conservative fallback	4.2 [1.2, 8.0]	97.5%	confirmed
F3 (estimate)	Pooled service, OR-Tools capped with best-screened fallback	94.4 [90.1, 97.1]	95%	–
F4 (estimate)	Pooled service, HGS capped with best-screened fallback	94.9 [91.0, 97.1]	95%	–

Screening admitted nothing on 9 instances for the OR-Tools capped procedure and on 8 for the HGS capped procedure. With their stored fallbacks the two procedures pooled 87.1% and 90.7%; with the best-screened fallback they pooled 94.4% and 94.9%. Both primary hypotheses met their decision criteria. The pooled estimates stay just below 95%. One instance cost more than the whole shortfall: there the OR-Tools anchor solves that fix the scaling ranges found no plan, so neither procedure could run and the instance scores zero, as the failure policy requires. On the other 52 instances the two procedures with the best-screened fallback pooled 96.3% and 96.7%; these figures are descriptive and were not prespecified. Hardware and the course of the run are described in Sections S9 and S11.

S10.15 Further checks

Appendix S14 reports three further exploratory checks. Repeating the selection with a Bonferroni bound over the 13 buffer levels removed all 11 admitted plans that later missed the validation bound, while one newly selected plan fell below it, at the cost of 12 admissions and at most 1.8 points of pooled service. Screening a candidate took 3 to 24 ms, against seconds for generating it. Under four other delay models, the advantage of the capped procedures over conservative speed held for fixed plans. When the stored candidates were screened again under each model, the procedure with its stored fallback fell up to 19.0 points below the fixed plans, while with the best-screened fallback of Section S10.14 it pooled between 93.2 and 96.7% (Appendix S14.3).

Section S13 reports the results of the first two confirmatory studies by size (Tables S21 and S22, descriptive only), every robust setting (Table S23), customer outcomes on the matched subset (Table S24), absolute planned resources (Table S25), service-threshold sensitivity (Table S26), the instance-level failure audit (Table S27), and the equal-budget deterministic benchmark against PyVRP/HGS (Table S28).

S11 Reproducibility and implementation checks

The study archive (Ali, 2026d) contains the candidate generators, the scenario-bank code, the test suite, the experiment drivers, the raw JSON records and the scripts that produce the results tables and the figures. The tests cover FIFO behaviour, lognormal moments, common random numbers that do not depend on evaluation

order, Wilson calculations, visit and capacity feasibility, the rule that contracted candidates are scored against the original windows, and the robust feasibility checks, which are compared with brute-force enumeration of deviation sets. Each run records the Python, NumPy and OR-Tools versions, the operating system, the solver budget, the seeds and the scenario counts.

The clock convention matters for reproduction. Relative time zero is 06:00, and the evaluator adds 360 minutes before looking up the full-day speed profile; leaving out that offset would evaluate a 00:00–10:00 shift instead. The vectorized evaluator is checked against the original scalar implementation, including period boundaries, customer lateness and shared arc disturbances.

Seeds are fixed. Admission uses seed 100000 plus the instance seed, all validation uses 5000 fresh scenarios from seed 400000 plus the instance seed, and the 1000-scenario disturbance sweep uses seed 500000 plus the instance seed. The capped, screened-conservative and clock-aware configurations were specified before they were run on the development instances, but earlier aggregate results motivated them, so they are exploratory and are neither preregistered nor an external replication; the fresh scenario draws do not remove the reuse of the instance set. Plans in the sweep are not re-screened when σ or ρ changes.

The first confirmatory protocol (Section S6) was publicly timestamped, with a hash manifest of the code that runs and analyses it, before its instances were solved; its results are in Section S10.7. One pipeline smoke test, on an instance outside the confirmatory set, was run while the protocol was being prepared, and the protocol discloses it as pilot data. The confirmatory study used only frozen scripts. The weight-sensitivity analysis was run after the freeze with a script outside the frozen file list. Its first run stopped on a coding error that affected only that script, and the corrected script produced the reported results. The second confirmatory study (Section S7) has its own frozen protocol and manifest. Its OR-Tools half uses the frozen scripts of the first study, and its HGS half is deterministic given the scaling ranges. The script that writes its tables and text stops unless its verdicts and counts equal the saved output of the frozen analysis and its estimates agree with that output to its two-decimal rounding. The one-decimal figures in the paper are rounded from the exact estimates, and those of the first study are produced in the same way. The development robust comparison (Section S8) was run after both studies with code outside the frozen file lists. Its search stops after a fixed number of iterations with a fixed seed, so its plans are deterministic, and the script that writes its tables and figure checks that every record is present and complete. The third protocol (Section S9) and its manifest were committed publicly and archived (Ali, 2026c) before any of its runs; part B started one minute after the release. Its OR-Tools half was interrupted once by a restart of the machine; the frozen drivers resume from the records already written, so completed instances were kept and the remaining ones were run afterwards. Both analysis programs ran once in confirmatory mode, after all records were complete.

All results tables and both figures are regenerated from the retained records. Internal planning buffers are always evaluated against the offered windows, candidate selection and final assessment use independent banks, and solver-quality comparisons between OR-Tools and HGS use equal wall-clock budgets on identical deterministic instances. These rules stop diagnostics and within-run selection from using their own validation outcomes. The robust search is compared with HGS at fixed iteration counts. The family-wise, timing and misspecification checks (Section S14), the post hoc fallback comparison, the fleet-matched robust contrast, the weight bracketing and the Gehring–Homberger check also use code outside the frozen lists. The family-wise script first repeats every stored selection with the original bound and stops unless all of them match, and the evaluator for the alternative delay models reproduces the stored validation results exactly when given the paper's model and validation seeds. A test checks this on OR-Tools, HGS and robust plans of every instance size.

Reproducibility shows that the reported numbers follow from the archived code and records. It does not show that the synthetic instances or the assumed disturbance model represent a real operation.

S12 Origin of the weight vector

The baseline weights were first computed from an Analytic Hierarchy Process matrix. They are used in this paper as a fixed experimental vector, and this appendix records their origin only. Criteria are ordered as distance cost, slack deficit, driver duration and fleet use. Entry a_{kl} is the judged importance of criterion k relative to l on Saaty's 1–9 scale:

$$\mathbf{A} = \begin{pmatrix} 1 & 3 & 5 & 7 \\ 1/3 & 1 & 3 & 5 \\ 1/5 & 1/3 & 1 & 2 \\ 1/7 & 1/5 & 1/2 & 1 \end{pmatrix}.$$

Its principal eigenvalue is $\lambda_{\max} = 4.0685$, and the normalised principal eigenvector is

$$\mathbf{w} = (0.5693,\ 0.2643,\ 0.1055,\ 0.0609).$$

With Saaty's random index 0.90 for four criteria, the consistency index is $(4.0685 - 4)/3 = 0.0228$ and the consistency ratio is 0.0254. As stated in Section S2, the third row and column were set when the third criterion was total distance and were not re-elicited for driver duration.

S13 Additional outcomes and failure records

Table S21. First confirmatory study by size (descriptive). For each procedure: the number of instances admitted by screening, with the number that kept the 0.95 lower bound on the fresh bank in parentheses, and the mean validated service-event probability (%) over all instances, including fallbacks.

n	Inst.	Original		Screened conservative		Capped	
20	20	17/20 (17)	90.2	15/20 (15)	87.8	17/20 (17)	90.2
50	15	8/15 (8)	74.6	8/15 (8)	80.3	12/15 (12)	87.4
100	10	4/10 (3)	71.3	3/10 (3)	80.0	10/10 (9)	97.1
200	8	3/8 (3)	71.5	2/8 (2)	75.4	7/8 (7)	92.4

Table S22. Second confirmatory study by size (descriptive): instances admitted by the capped procedure, and mean validated service-event probability (%) over all instances, including fallbacks.

n	Inst.	OR-Tools capped			HGS		
		Admitted	Slack fb.	Cons. fb.	Admitted	Conservative	Capped, cons. fb.
20	20	13	83.8	83.3	14	56.8	85.8
50	15	14	93.7	95.7	13	61.9	92.4
100	10	10	97.9	97.9	10	64.1	97.4
200	8	8	97.5	97.5	8	48.9	97.1

Table S23. Every robust setting on the 45 development instances: instances with a plan, mean validated service-event probability (%) by size and pooled, mean distance relative to the nominal plan and mean vehicles over instances with a plan. A missing plan scores zero.

Model	θ	Γ	Plans	$n = 20$	$n = 50$	$n = 100$	Pooled	Dist. (%)	Veh.
Nominal	0	0	45	20.7	17.7	26.4	21.0	0.0	5.98
Standard	0.2	1	41	47.3	37.9	40.8	42.7	1.6	6.02
Standard	0.2	2	41	58.6	47.4	48.9	52.7	2.2	6.17
Standard	0.2	3	41	63.2	50.3	54.6	57.0	2.6	6.34
Standard	0.2	5	41	65.2	53.8	57.8	59.8	2.9	6.37
Standard	0.2	box	41	65.3	54.2	59.8	60.4	3.0	6.32
Standard	0.4	1	37	70.8	54.5	35.4	57.5	3.4	6.00
Standard	0.4	2	37	86.0	67.2	43.6	70.3	5.8	6.19
Standard	0.4	3	37	89.2	70.7	46.2	73.5	6.6	6.46
Standard	0.4	5	37	90.5	72.2	47.6	74.9	6.9	6.57
Standard	0.4	box	37	90.3	72.5	47.9	75.0	7.0	6.54
Standard	0.6	1	29	82.5	39.0	25.3	55.3	5.6	5.62
Standard	0.6	2	29	91.6	44.8	29.2	62.2	9.1	6.31
Standard	0.6	3	29	92.7	45.8	29.6	63.1	9.9	6.38
Standard	0.6	5	29	92.8	46.1	29.9	63.2	10.7	6.52
Standard	0.6	box	29	93.0	46.1	29.9	63.3	10.8	6.59
Relief	0.2	1	45	47.3	43.0	50.5	46.6	1.7	6.22
Relief	0.2	2	45	58.6	54.0	61.2	57.6	2.3	6.36
Relief	0.2	3	45	63.2	57.6	68.0	62.4	2.7	6.53
Relief	0.2	5	45	65.2	62.2	71.5	65.6	2.9	6.58
Relief	0.2	box	45	65.3	62.4	74.1	66.3	3.1	6.51
Relief	0.4	1	45	70.8	66.1	70.1	69.1	3.7	6.53
Relief	0.4	2	45	86.0	83.3	87.2	85.4	6.3	6.89
Relief	0.4	3	45	89.2	87.5	92.5	89.4	7.1	7.18
Relief	0.4	5	45	90.5	90.2	94.7	91.4	7.4	7.24
Relief	0.4	box	45	90.3	90.0	95.7	91.4	7.6	7.31
Relief	0.6	1	45	86.9	82.2	84.9	84.9	6.6	6.87
Relief	0.6	2	45	96.5	95.2	97.3	96.2	10.3	7.64
Relief	0.6	3	45	97.6	97.5	98.7	97.8	11.4	7.78
Relief	0.6	5	45	97.7	98.3	99.3	98.2	12.2	7.96
Relief	0.6	box	45	97.8	98.4	99.4	98.4	12.4	8.04

Table S24. Customer outcomes on the instances where conservative-speed planning returned a plan. The clock approximation also excludes the instances where it found no schedule, and the counts show this.

Configuration	k	On time (%)	Expected late
Nominal	46	83.6 [82.5, 84.6]	10.00 [7.95, 12.20]
Conservative speed	46	93.9 [93.4, 94.4]	3.47 [2.84, 4.14]
Slack-aware	46	90.9 [90.0, 91.7]	5.46 [4.38, 6.65]
Fleet-matched	46	83.0 [81.9, 84.0]	10.47 [8.29, 12.87]
Original procedure	46	96.6 [95.1, 97.9]	2.20 [1.26, 3.31]
Screened conservative	46	97.1 [96.3, 97.8]	2.20 [1.47, 2.99]
Capped procedure	46	97.7 [96.4, 98.9]	0.91 [0.56, 1.32]
Clock approximation	44	88.0 [87.2, 88.7]	6.73 [5.39, 8.23]

Table S25. Mean absolute planned resources on the same instances as Table S24: distance in km, scheduled duration in minutes, and vehicles. These are not claims of optimality.

Configuration	k	Distance	Duration	Fleet
Nominal	46	1180.7 [989.9, 1381.0]	2906.5 [2379.9, 3481.3]	7.67 [6.46, 8.98]
Conservative speed	46	1216.2 [1021.5, 1422.0]	3352.5 [2738.5, 4011.0]	8.30 [6.93, 9.76]
Slack-aware	46	1196.3 [1003.0, 1404.0]	2902.0 [2380.3, 3466.2]	7.46 [6.22, 8.78]
Fleet-matched	46	1174.2 [989.2, 1372.8]	2833.6 [2330.6, 3382.6]	7.46 [6.22, 8.76]
Original procedure	46	1247.9 [1049.3, 1453.0]	3171.2 [2620.9, 3777.3]	8.17 [6.89, 9.57]
Screened conservative	46	1252.8 [1056.4, 1461.2]	3494.5 [2874.0, 4161.0]	8.76 [7.35, 10.28]
Capped procedure	46	1283.5 [1076.5, 1506.8]	3317.3 [2709.4, 3964.1]	8.52 [7.13, 10.04]
Clock approximation	44	1137.2 [955.5, 1330.9]	2868.3 [2335.0, 3456.8]	7.64 [6.32, 9.11]

Table S26. Sensitivity to the service threshold κ on the 46 common instances, with the same plans and the same fresh validation bank in every column. At $n = 20/50/100/200$, the number of customers allowed to be late is $2/5/10/20$ at $\kappa = .90$, $1/2/5/10$ at .95, $0/1/2/4$ at .98, and zero at 1.00.

κ	N	C	S	P	SC
0.90	38.8 [36.5, 41.2]	80.6 [78.8, 82.4]	68.8 [65.9, 71.7]	89.2 [84.2, 93.7]	91.2 [88.5, 93.8]
0.95	21.9 [20.3, 23.5]	65.6 [63.1, 68.0]	52.8 [49.9, 55.8]	83.1 [76.3, 89.4]	82.8 [77.9, 87.6]
0.98	11.2 [10.3, 12.2]	48.2 [45.6, 50.8]	36.4 [33.8, 39.0]	75.9 [68.2, 83.3]	73.2 [67.1, 79.3]
1.00	6.4 [5.4, 7.4]	34.4 [31.8, 37.0]	26.1 [23.8, 28.5]	68.2 [59.6, 76.2]	62.4 [53.8, 70.9]

S13.1 Instance-level failure audit

Table S27. Every instance on which the original procedure admitted no plan. Best β and the lower confidence bound (LCB) refer to the best returned candidate; the no-plan β is the first buffer at which the solver returned no plan; the proof β is the first buffer with a reachability certificate. Unresolved marks at least one no-plan result without a certificate. A dash means none up to 60 minutes.

n	Seed	Best β	LCB (%)	No-plan β	Proof β	Unresolved
20	7200	45	90.7	50	–	Yes
20	7201	35	93.5	40	–	Yes
20	7210	35	90.8	45	–	Yes
20	7212	5	37.9	10	–	Yes
20	7216	55	90.7	60	60	No
20	7218	10	65.9	15	50	Yes
50	7501	25	87.5	30	35	Yes
50	7503	10	64.1	15	15	No
50	7504	35	91.3	40	40	No
50	7506	0	44.7	5	5	No
50	7508	35	85.9	40	40	No
50	7511	35	88.5	40	50	Yes
50	7513	40	93.6	45	60	Yes
100	8003	20	85.3	25	25	No
100	8004	15	81.5	20	20	No
100	8007	5	73.6	15	15	No
100	8008	25	91.7	35	35	No
200	9004	10	71.2	15	15	No
200	9005	20	88.9	25	25	No
200	9006	25	94.9	40	40	No
200	9007	20	86.4	25	25	No

Table S28. Retained equal-budget benchmark on the deterministic core: excess distance of OR-Tools routes over PyVRP/HGS routes (%). Counts are valid pairs at the one-second and five-second budgets; one pair is missing at one second.

n	Valid pairs (1s/5s)	One second	Five seconds
20	19/20	1.0 [0.3, 1.8]	0.0 [0.0, 0.0]
50	15/15	4.8 [3.3, 6.2]	3.0 [1.9, 4.2]
100	10/10	8.5 [5.9, 11.1]	6.3 [4.7, 7.9]
200	8/8	32.9 [29.4, 36.2]	11.1 [9.3, 13.5]

S14 Further exploratory checks

The three checks in this appendix were added after the first two confirmatory studies and are exploratory. The first reuses the stored screening counts, the second times screening and one HGS solve on each development instance and counts the selected buffers, and the third evaluates the stored plans on new scenarios. The misspecification check also reruns the selection under each alternative model.

S14.1 Family-wise screening

The admission bound holds for each candidate separately, while the procedure searches 13 buffer levels per instance. To check whether this matters, every stored candidate family was screened again with the Wilson bound at $\eta/13$, a Bonferroni adjustment that makes the 95% bound hold simultaneously over the grid. Selection and fallback were otherwise unchanged, and any newly selected plan was validated on the original fresh bank (Table S29). Over the 8 combinations of study and procedure, the adjustment reduced the admitted instances from 318 to 306, at most 2 per combination, and selected a different plan on 79 of the instances it still admitted. Admitted plans whose validation lower bound fell below 0.95 went from 11 of 318 to 1 of 306. Pooled service changed by between -1.8 and 0.2 points. Admitted plans averaged 97.3 to 98.0% on the fresh bank under the per-candidate bound, depending on study and procedure, and 97.7 to 98.3% under the family-wise bound. The stricter bound thus removed all 11 admitted plans whose validation bound fell below 0.95, while one plan it newly selected fell below it, at the cost of a few admissions and up to 1.8 points of pooled service.

Table S29. Family-wise screening (exploratory). Adm.: instances with an admitted plan. Kept: admitted plans whose validation lower bound was at least 0.95. Pooled: validated service-event probability (%) of the procedure over all instances, fallbacks included. Columns marked FW use the Bonferroni bound over the 13 buffer levels.

Study	Procedure	Adm.	Kept	Pooled	Adm. FW	Kept FW	Pooled FW
Development	Original (OR-Tools)	32	30	78.9	30	30	77.1
Development	Capped (OR-Tools)	42	40	86.6	40	40	85.0
Development	HGS capped, conservative fallback	48	46	94.2	46	46	93.2
Confirmatory 1	Original (OR-Tools)	32	31	79.4	30	30	77.8
Confirmatory 1	Capped (OR-Tools)	46	45	91.0	45	45	90.7
Confirmatory 2	Original (OR-Tools)	28	28	77.1	27	27	76.2
Confirmatory 2	Capped (OR-Tools)	45	45	91.4	45	44	91.6
Confirmatory 2	HGS capped, conservative fallback	45	42	91.6	43	43	90.6

S14.2 Computation time and buffer choice

Screening is cheap relative to generation. On one x86-64 Linux machine, evaluating one candidate on the 1000-scenario screening bank took a median of 3 ms at 20 customers and 24 ms at 200. A candidate solve has a fixed five-second OR-Tools budget, and a median HGS solve took 2.2 s at 20 customers and 9.3 s at 200. The procedure's cost therefore lies in up to 13 candidate solves ($\beta = 0$ to 60 minutes), each of which OR-Tools repeats at most twice if it returns no plan. Absolute times depend on the machine.

Among admitted selections, the buffer grid was rarely binding for OR-Tools. Over the three studies, 191 of the 225 OR-Tools selections used a buffer between 25 and 45 minutes, and 4 used the largest buffer of 60

minutes; the median selected buffer was 35 minutes. HGS selected larger buffers (median 50 minutes), and 11 of its 93 selections were at 60 minutes, so a wider grid could change some HGS selections. On instances where nothing was admitted, no buffer up to 60 minutes passed, and the grid cannot show whether a larger one would have.

S14.3 Misspecified delay models

The plans were screened under one assumed disturbance model. To test how much the conclusions depend on it, the stored plans of the development study and the first two confirmatory studies were evaluated, without re-screening or re-optimization, on 5000 new scenarios per instance under four other models. The models were chosen before the run, but they were not registered. The heavy-tail model replaces the normal common and local log-errors by Student-t errors with three degrees of freedom, scaled to the same variance. At equal variance these errors are more peaked near zero and have heavier tails, so the model changes the shape of the distribution more than its spread; the multipliers then have infinite mean, although every simulated value is finite. The incident model adds, on each arc traversal, an independent 3% chance that the travel time doubles. The time-of-day model uses $\sigma = 0.30$ and $\rho = 0.8$ for departures in the morning peak (07:00–09:00) and $\sigma = 0.15$ and $\rho = 0.3$ at other times, in place of $\sigma = 0.20$ and $\rho = 0.5$ throughout. (The code also raises the noise in the evening peak, which begins after every time window has closed and so has no effect.) The stronger-peak model divides speed by 1.4 between 07:00 and 09:00, where the plans assumed 1.2. Tables S30 and S31 pool the 159 OR-Tools instances, the 106 HGS instances and the 45 instances of the robust comparison.

Table S30. Misspecified delay models (exploratory): pooled service-event probability (%) of fixed plans on 5000 new scenarios per instance. OR-Tools columns use 159 instances, HGS columns 106 and screened robust with relief 45; because the instance sets differ, configurations are compared in Table S31. Cons.: conservative speed. Original and Capped: the OR-Tools procedures. HGS capped: the HGS capped procedure with conservative fallback. A missing plan scores zero.

Delay model	Cons.	Original	Capped	HGS cons.	HGS capped	Scr. robust
Paper's model (new scenarios)	57.8	78.5	89.7	59.6	92.9	97.2
Heavy tails	64.0	79.8	89.9	65.8	93.6	97.3
Incidents	49.6	71.7	84.6	51.1	87.5	92.2
Time-of-day noise	56.4	77.6	88.4	58.3	91.3	93.5
Stronger morning peak	36.8	67.1	82.9	38.4	86.0	90.8

Table S31. Misspecified delay models (exploratory): paired differences in service probability (percentage points) with 95% bootstrap intervals over instances. Cons.: conservative speed.

Delay model	Original minus cons.	Capped minus cons.	HGS capped minus HGS cons.	Scr. robust minus HGS capped
Paper's model (new scenarios)	20.7 [16.7, 24.7]	31.9 [28.3, 35.6]	33.3 [29.0, 38.0]	3.6 [0.7, 7.0]
Heavy tails	15.8 [11.5, 20.0]	25.8 [21.9, 29.8]	27.8 [23.3, 32.7]	3.1 [0.8, 5.7]
Incidents	22.1 [18.1, 26.1]	35.0 [31.5, 38.5]	36.4 [32.3, 40.7]	4.5 [1.2, 8.4]
Time-of-day noise	21.2 [17.4, 25.0]	32.0 [28.5, 35.5]	32.9 [28.7, 37.5]	1.7 [−1.0, 5.1]
Stronger morning peak	30.3 [25.5, 35.1]	46.1 [42.0, 50.1]	47.7 [43.0, 52.3]	3.6 [−1.0, 9.1]

Heavy tails did not lower service. Conservative speed gained most, from 57.8 to 64.0% for OR-Tools, and the capped procedures and screened robust changed by less than one point. The other three models lowered service for every configuration, most of all the stronger peak and then incidents. The stronger peak reduced the capped procedures by up to 6.9 points and conservative speed by up to 21.2 points: travel in the morning peak now takes 40% longer than at nominal speed, which the 20% pad no longer covers. The advantage over conservative speed held under every model, with lower interval limits of at least 11.5 points for the original procedure, 21.9 for the capped procedure and 23.3 for the HGS capped procedure. On its 45 instances, screened robust with relief stayed ahead of the HGS capped procedure under every model, but its lead fell from 3.6 to 1.7 points under time-of-day noise, and the intervals include zero under time-of-day noise and the stronger peak. These results concern fixed plans.

Re-screening asks what the procedure itself would do under each model. Every stored candidate of the capped families was screened again on 1000 scenarios drawn from that model (seed 100000 plus the instance seed), the selection rule was applied unchanged, and the selected plan was validated on the same 5000 scenarios of that model. The candidate families were not regenerated. Under the paper's model this reproduces every stored selection, which the script checks. Table S32 reports two fallbacks for instances where nothing is admitted: the procedure's stored fallback, and the candidate with the highest screening success (Section S10.14).

Table S32. Re-screening under each delay model (exploratory): pooled service-event probability (%) of the capped procedures over 159 OR-Tools and 106 HGS instances. Fixed: the stored plans. Re-screened: the stored candidates screened again under the model, with the stored fallback. Best: the same, with the best-screened fallback.

Delay model	OR-Tools capped			HGS capped		
	Fixed	Re-screened	Best	Fixed	Re-screened	Best
Paper's model (new scenarios)	89.7	89.7	96.0	92.9	92.9	96.7
Heavy tails	89.9	91.3	96.0	93.6	93.8	96.5
Incidents	84.6	69.4	93.2	87.5	68.5	93.7
Time-of-day noise	88.4	86.8	95.4	91.3	88.5	96.1
Stronger morning peak	82.9	72.8	93.5	86.0	72.4	94.8

With the stored fallback, re-screening did worse than keeping the fixed plans under incidents and the stronger peak, by up to 19.0 points, and for HGS also under time-of-day noise. Fewer candidates passed under these models (OR-Tools 81 and 102 admissions against 133 under the paper's model), and the instances that failed received the weak stored fallback. With the best-screened fallback, re-screening exceeded the fixed plans under every model, and pooled service stayed between 93.2 and 96.0% for OR-Tools and between 93.7 and 96.7% for HGS.

References

Agra, A., Christiansen, M., Figueiredo, R., Hvattum, L.M., Poss, M., Requejo, C., 2013. The robust vehicle routing problem with time windows. *Comput Oper Res* 40, 3, 856–866.

Ali, S., 2026a. Reliability-screened routing: research code and evidence archive. Zenodo, https://doi.org/10.5281/zenodo.23185705. Confirmatory protocol 1 freeze, commit 41fae67d9520bd5dd01fcac87e680a5a69325d10.

Ali, S., 2026b. Reliability-screened routing: research code and evidence archive. Zenodo, https://doi.org/10.5281/zenodo.23208320. Confirmatory protocol 2 freeze, commit 3ba6b71288a2a51872ab6a877fa86eb28ff7671d.

Ali, S., 2026c. Reliability-screened routing: research code and evidence archive. Zenodo, https://doi.org/10.5281/zenodo.23242687. Confirmatory protocol 3 freeze, commit 21ffaa8eed4e899778e14014b73533fd4d14d865.

Ali, S., 2026d. Reliability-screened routing: research code and evidence archive. Zenodo, https://doi.org/10.5281/zenodo.23265196. Version v3.0, software and dataset.

Bertsimas, D., Sim, M., 2004. The price of robustness. *Oper Res* 52, 1, 35–53.

Gehring, H., Homberger, J., 1999. A parallel hybrid evolutionary metaheuristic for the vehicle routing problem with time windows. In Miettinen, K., Mäkelä, M. and Toivanen, J. (eds), *Proceedings of EUROGEN99*, University of Jyväskylä, Jyväskylä, pp. 57–64.

ML4VRP, 2024. ML4VRP 2024 competition: instances and evaluator. GitHub, https://github.com/ML4VRP/ML4VRP2024. Instances/CVRPTW. Accessed 6 October 2026.

Saaty, T.L., 1980. *The Analytic Hierarchy Process*. McGraw-Hill, New York.

Solomon, M.M., 1987. Algorithms for the vehicle routing and scheduling problems with time window constraints. *Oper Res* 35, 2, 254–265.

Vidal, T., 2022. Hybrid genetic search for the CVRP: Open-source implementation and SWAP* neighborhood. *Comput Oper Res* 140, 105643.

Wouda, N.A., Lan, L., Kool, W., 2024. PyVRP: A high-performance VRP solver package. *INFORMS J Comput* 36, 4, 943–955.